

Digital Contracting Service

Technisches Handbuch

Stand 2026-07-31

DCS Technisches Handbuch

Dieses Handbuch erklärt den Digital Contracting Service (DCS) auf Systemebene: Aufbau, Abläufe, Schnittstellen und Betriebsverhalten. Zielgruppe sind Entwickler, Betreiber und Integratoren, die das System verstehen, betreiben oder anbinden wollen. Quelle der Wahrheit ist der Code.

Inhaltsverzeichnis

Kapitel	Inhalt
01 Systemüberblick	Was das DCS ist, Einordnung in Gaia-X/XFSC/FACIS, Kernfähigkeiten
02 Architektur	Komponentenlandkarte, Kommunikationsbeziehungen, Gesamtdiagramm
03 Vorlagen und Vertragsinhalte	Template-Lifecycle, Semantic Hub (JSON-LD/SHACL/ODRL), Klausel-Katalog
04 Vertragslebenszyklus	State-Machine, Rollen/Tasks/RBAC, Verhandlung, lokal vs. grenzüberschreitend
05 Identität und Authentifizierung	Endnutzer (OID4VP/SD-JWT), Maschinen-Identitäten (Hydra), Instanz (did:web + HSM), Issuer-Vertrauen
06 Signaturen	Wallet-Ceremony, AES/QES, PAdES/JAdES, DSS, TSA, PID
07 Dokumente und Provenance	pdf-core, deterministisches Rendering, C2PA, IPFS, Verschlüsselung at rest
08 Föderation	Instanz-zu-Instanz-Austausch, Trust-Modell, Federated Catalogue
09 Audit und Compliance	Audit-Trail, Merkle-Checkpoints, Workflow-Gates, Monitoring, ODRL/OPA, Reports
10 Betrieb und Monitoring	Betriebsverhalten im Normal- und Fehlerfall: Probes, Hintergrundprozesse, Signale, Incidents
Anhang A Schnittstellenreferenz	API-Gruppen, Events, well-known-Endpunkte
Anhang B Entscheidungsarchiv	Verweisliste auf die Architecture Decision Records (ADRs)

Abgrenzung zu den übrigen Dokumenten

Dieses Handbuch erklärt. Es enthält bewusst keine Befehle, keine Wertetabellen und keine Schritt-für-Schritt-Prozeduren.

Frage	Dokument
Wie ist das System gebaut, wie läuft ein Vorgang durch, was bedeutet ein Signal?	Dieses Handbuch
Wie installiere, konfiguriere, aktualisiere und deinstalliere ich eine Instanz?	<code>docs/deployment-guide.md</code>
Wie sichere und stelle ich Daten wieder her?	<code>docs/backup-integration-guide.md</code>
Wie bediene ich das System als Fachanwender?	<code>docs/user-manual/</code>
Warum wurde etwas so entschieden?	Die ADRs unter <code>docs/</code> , siehe Anhang B

Konkrete Werte, Befehle und Konfigurationsreferenzen stehen an genau einer Stelle, nämlich im Deployment-Leitfaden. Wo dieses Handbuch einen solchen Wert bräuchte, verweist es dorthin.

Leseempfehlung

- **Neu im Projekt:** Kapitel 01 und 02, danach Kapitel 04 mit dem zentralen Ablauf des Systems.
- **Betreiber:** Kapitel 02, dann Kapitel 10; Kapitel 09 für Audit-Trail und Incident-Sicht. Ausführbare Verfahren stehen im Deployment-Leitfaden.
- **Integrator (Zielsysteme, Peer-Instanzen):** Kapitel 02, 07, 08 und Anhang A.
- **Sicherheits-/Compliance-Review:** Kapitel 05, 06, 09 und Anhang B.

01 Systemüberblick

Was das DCS ist

Der **Digital Contracting Service (DCS)** ist ein föderiertes, cross-federation-fähiges und on-premise hostbares System für die sichere B2B-Kommunikation, Verhandlung und automatisierte Verarbeitung digitaler Verträge. Der zentrale Gedanke: **Ein digitaler Vertrag ist nicht nur ein signiertes Dokument, sondern eine interoperable, maschineninterpretierbare und kryptographisch nachweisbare Vereinbarung zwischen Organisationen.**

Jede Organisation betreibt ihre eigene DCS-Instanz. Instanzen unterschiedlicher Betreiber tauschen Vorlagen und Verträge aus, ohne einander implizit zu vertrauen. Vertrauen wird pro Interaktion kryptographisch hergestellt und geprüft.

Einordnung: Gaia-X, XFSC, FACIS

Das DCS entsteht im Rahmen von **FACIS** (Federation Architecture for Composed Infrastructure Services) im Eclipse-XFSC-Ökosystem und fügt sich in die Gaia-X-Infrastrukturlandschaft ein:

- **Federated Catalogue:** Freigegebene Vorlagen werden als Self-Descriptions registriert und damit föderationsweit auffindbar.
- **ORCE (XFSC Orchestration Engine):** Node-RED-basierte Orchestrierungsschicht für Zeitstempel, Archiv-Notariat, Zielsystem-Anbindung, Trust-PDP und die auslagerbaren Prüf- und Freigabeschritte des Compliance-Subsystems.
- **XFSC Status List Service:** Widerrufsstatus der vom DCS ausgestellten Lifecycle-Credentials.
- **EUDI Wallet / eIDAS 2.0:** Endnutzer authentifizieren sich über Verifiable Credentials (OpenID4VP); Signaturen folgen den eIDAS-Formaten PAdES/JAdES.

Verbindliche Anforderungsbasis ist die SRS des FACIS-DCS; die Positionierung gegenüber den XFSC-Komponenten beschreibt ADR-5.

Kernfähigkeiten

Semantische Vorlagen, maschinenlesbare Bedingungen. Organisationen definieren auf Basis frei wählbarer Ontologien maschinenlesbare Vertragsvorlagen. Ein instanzlokaler Semantic Hub versioniert die JSON-LD-Kontexte, SHACL-Shapes und Validierungsprofile, gegen die jedes Dokument erzeugt und geprüft wird, und stellt den Klausel-Katalog bereit. Vertragsbedingungen sind mit ODRL 2.0 formal beschrieben und damit für Menschen und Maschinen interpretierbar.

Verhandlung und formalisierter Abschluss. Aus Vorlagen erzeugte Verträge durchlaufen eine definierte State-Machine von Entwurf über Angebot, Verhandlung, Review und Freigabe bis zur Signatur. Einzelne Bedingungen werden als Change Requests verhandelt. Jede Instanz führt ihren eigenen Workflow mit eigenen Rollen; über die Instanzgrenze wandert der Vertrag selbst, nicht der interne Zustand der Gegenseite.

eIDAS-konforme Signatur und Übergabe an Zielsysteme. Abgeschlossene Verträge sind elektronisch signierte, eIDAS-2.0-konforme und zugleich maschinenlesbare Dokumente: Das PDF/A-3 trägt die kanonische JSON-LD als eingebetteten Anhang und wird per PAdES/JAdES signiert. Zielsysteme setzen vereinbarte Bedingungen automatisiert um oder werten sie aus und melden Zustände, Nachweise und KPI-Werte an die beteiligten Instanzen zurück. Der Vertrag bleibt so über seinen gesamten Lebenszyklus eine maschineninterpretierbare Vereinbarung.

Zero-Trust-Identitätsmodell. Vertrauen entsteht aus kryptographisch verifizierbaren Identitäten, Credentials, Signaturen, Vertrauenslisten und unabhängig nachvollziehbaren Prüfprozessen, nicht aus Netzwerkposition oder organisatorischer Zugehörigkeit. Endnutzer melden sich per Verifiable Presentation an (OpenID4VP, SD-JWT VC, z. B. aus einer EUDI Wallet); maschinelle Aufrufer erhalten eigene OAuth2-Clients, deren Rechte in einer Registry der Instanz stehen, nie in Token-Claims; jede Instanz besitzt eine `did:web`-Identität, deren private Schlüssel ausschließlich in einem PKCS#11-Token (HSM) liegen. Ausstellervertrauen ist zweckgebunden und organisationsgebunden (ADR-31), und jede Föderationsinteraktion konsultiert fail-closed einen lokalen Policy-Endpunkt.

Kryptographisch nachvollziehbare Auditierbarkeit. Vertrags-, Verhandlungs-, Signatur- und Zustandereignisse landen in einem manipulationsnachweisbaren Audit-Trail: hash-verkettete Einträge je Ressource, gebündelt zu Merkle-Checkpoints, deren Root einen RFC-3161-Zeitstempel erhält und in IPFS verankert wird. So lässt sich unabhängig belegen, dass eine Vertrags- oder Audit-Repräsentation zu einem bestimmten Zeitpunkt in einer bestimmten Form existiert hat und nachträglich nicht unbemerkt verändert wurde.

Unabhängige Validierung: maschinenlesbar gegen menschenlesbar. Der Dokumentendienst pdf-core rendert deterministisch: Dieselbe JSON-LD-Payload erzeugt byte-identischen Seiteninhalt. Jede beteiligte Instanz extrahiert aus einem eingehenden signierten Dokument die eingebettete JSON-LD, rendert sie unabhängig neu und vergleicht das Ergebnis mit dem erhaltenen Dokument. Keine Instanz ist dafür auf die Aussage der ausstellenden oder übermittelnden Instanz angewiesen. Eine C2PA-Provenance-Kette im Dokument macht zusätzlich die Herkunft des sichtbaren Inhalts nachweisbar.

Vertraulichkeit und Löschbarkeit. Jedes in IPFS abgelegte Artefakt ist verschlüsselt. Der Schlüssel ist pro Vertrag zufällig gezogen und im Ruhezustand nur durch das HSM der jeweiligen Instanz zu öffnen. Die Löschung eines Vertrags im Sinne von Art. 17 DSGVO ist die protokollierte Vernichtung dieser Schlüsselkopien auf beiden beteiligten Instanzen; die Merkle-Beweise des Audit-Trails bleiben gültig, weil sie nie vom lesbaren Inhalt abhingen (ADR-28, Kapitel 07 und 09).

Das DCS verbindet damit föderierte Identitäten, Verifiable Credentials, semantische Vertragsmodelle, ODRL-Bedingungen, interoperable Verhandlung, eIDAS-Signaturen, automatisierte Umsetzung und kryptographisch verankerte Auditierbarkeit in einer gemeinsamen Vertrauens- und Kommunikationsinfrastruktur. Wie diese Fähigkeiten auf Komponenten verteilt sind, zeigt Kapitel 02.

02 Architektur

Dieses Kapitel beschreibt die Komponentenlandkarte einer DCS-Instanz: welche Bausteine es gibt, welche Verantwortung sie tragen und wer mit wem spricht. Die fachlichen Abläufe behandeln die Kapitel 03 bis 09, das Betriebsverhalten Kapitel 10, die Installation der Deployment-Leitfaden.

Grundschnitt

Eine DCS-Instanz besteht aus drei selbst entwickelten Diensten (**Backend**, **pdf-core**, **Frontend**) und einer Reihe von Infrastruktur- und XFSC-Komponenten im selben Deployment. Das Backend ist ein **modularer Monolith**: Alle Fachdomänen laufen in einem Prozess, teilen sich eine PostgreSQL-Instanz und sind über einen internen Event-Bus (NATS, CloudEvents) entkoppelt. Eigene Prozesse existieren dort, wo eine andere technische Verantwortung beginnt: Byte-Arbeit am PDF in pdf-core, Orchestrierung und auslagerbare Prüfschritte in ORCE, AdES-Signaturvalidierung in DSS.

Zwei Schnitte prägen die Architektur:

1. **Alle privaten Schlüssel der Instanz liegen im PKCS#11-Token, und nur das Backend greift darauf zu** (ADR-1). Wer eine Signatur braucht, erhält sie vom Backend.
2. **Jedes dauerhaft gespeicherte Artefakt ist verschlüsselt, bevor es den Prozess verlässt** (ADR-28). Der Schlüssel ist an einen Löschbereich gebunden: pro Vertrag, pro Vorlage oder instanzweit.

Komponentenlandkarte

Selbst entwickelte Dienste

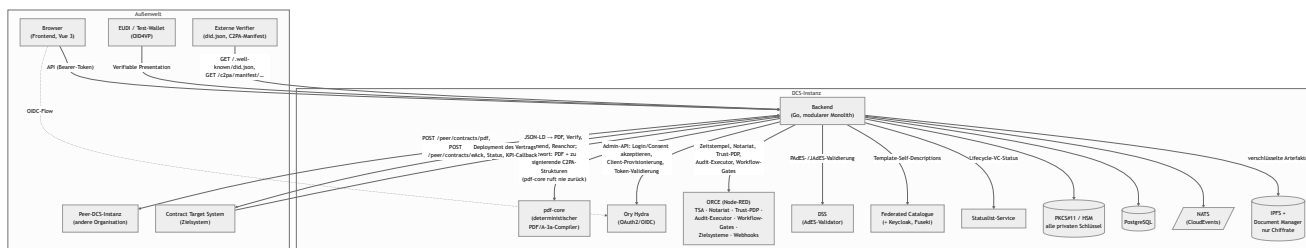
Komponente	Technologie	Verantwortung
Backend	Go, Goa v3 (design-first)	Gesamte Fachlogik: Template- und Vertragslebenszyklus, Verhandlung, Signaturmanagement, Föderation, Audit-Trail, Authentifizierung, Semantic Hub, Schlüsselinventar. Bedient die HTTP-API
pdf-core	Go	Deterministischer PDF/A-3a-Compiler: JSON-LD zu byte-stabilem PDF mit eingebetteter kanonischer JSON-LD und C2PA-Provenance-Kette; Re-Rendering-Verifikation, inkrementelle Amendments, Provenance-Re-Anchor. Hält kein Schlüsselmaterial: Es liefert die zu signierenden C2PA-Strukturen an das Backend zurück
Frontend	Vue 3, Vite, Pinia	Browser-UI; spricht ausschließlich die Backend-API und wird vom Backend mit ausgeliefert

Das Backend gliedert sich intern in Fachdomänen, die jeweils eine API-Gruppe tragen; die Endpunktübersicht liefert Anhang A.

Fremd- und Infrastrukturkomponenten

Komponente	Rolle im System
PostgreSQL	Transaktionale Persistenz aller Backend-Domänen plus separate Datenbanken für Hydra und den Federated Catalogue
NATS	Interner Event-Bus (CloudEvents). Trägt Domänenereignisse aus der transaktionalen Outbox an nachgelagerte Verbraucher
IPFS (+ Document Manager)	Content-adressierter Speicher für alle Artefakte; der Document Manager stellt eine mandantenfähige API davor. Das Backend legt dort ausschließlich Chifftrate ab
Ory Hydra	OAuth2/OIDC-Provider der Instanz: Tokens für Frontend-Nutzer (nach OID4VP-Login) und maschinelle Aufrufer
ORCE	XFSC Orchestration Engine (Node-RED): RFC-3161-TSA, Archiv-Notariat und Audit-Log-Senke, Trust-PDP, Checkpoint-Anker, Zielsystem-Anbindung, Event-Webhooks, Audit-Executor, Workflow-Gate-Ausführung
DSS	EU Digital Signature Service als externer AdES-Validator. Ohne ihn nimmt der Signatur-Submit-Pfad keine Signatur an
Federated Catalogue (+ Keycloak, Fuseki, eigene PostgreSQL)	XFSC-Katalog für die Veröffentlichung freigegebener Templates als Self-Descriptions; Keycloak stellt die Service-Account-Tokens
Statuslist-Service	Führt den Widerrufsstatus der vom DCS ausgestellten Lifecycle-Credentials. Die Statuslisten der Login-, Vollmachts- und PID-Credentials bedienen deren Aussteller selbst; eine unsignierte Statusliste wird nirgends akzeptiert (ADR-34)
PKCS#11-Token (HSM)	Hält sämtliche privaten Schlüssel der Instanz. Nur das Backend greift darauf zu
Ingress	Externe Erreichbarkeit; dahinter liegen die öffentlichen Auflösungspfade ebenso wie die authentifizierte API

Wer spricht mit wem



Die wichtigsten Beziehungen:

- **Frontend → Backend:** Der Browser spricht ausschließlich die Backend-API, authentifiziert per Hydra-Token nach OID4VP-Wallet-Login (Kapitel 05).
- **Backend ↔ pdf-core:** Das Backend lässt zu jeder relevanten Änderung das PDF kompilieren und eingehende Dokumente per Re-Rendering verifizieren. Für die C2PA-Manifestsignatur liefert pdf-core die zu signierenden Strukturen mit dem gerenderten PDF zurück; das Backend signiert sie mit dem HSM und lässt sie in einem zweiten Aufruf einbetten. pdf-core hält nie einen Signer (Kapitel 07).

- **Outbox → NATS → Verbraucher:** Jede fachliche Änderung schreibt ihr Ereignis in derselben Transaktion in eine Outbox. Ein Hintergrundprozess publiziert auf NATS (PDF-Regenerierung, Föderations-Versand, Webhooks, Auto-Deployment) und verankert audit-sichtbare Ereignisse als Merkle-committete Audit-Einträge (Kapitel 09).
- **Backend ↔ Peer:** Das signierte PDF ist das Wire-Format der Föderation; versendet wird in `OFFERED`, `NEGOTIATION`, `SIGNED` und `REVOKED`, geschützt durch did:web-Challenge-Response und das fail-closed Trust Gate (Kapitel 08). Fehlversuche landen in einer Retry-Tabelle.
- **Backend ↔ ORCE:** Zwei Randdienste sind harte Startabhängigkeiten, weil sie im Prüfpfad sitzen: der Workflow-Gate-Executor (Kapitel 04) und der Audit-Executor (Kapitel 09). Als Credential-Aussteller läuft ORCE zusätzlich in eigenen Releases: einer je Instanz für die Handlungsvollmacht (PoA), einer föderationsweit als PID-Aussteller (ADR-31).
- **Backend ↔ Federated Catalogue / Statuslist:** Ein konfigurierter Katalog ist eine harte Startabhängigkeit: Das Backend prüft ihn beim Start funktional und synchronisiert die Schemata. Der Statuslist-Service führt den Widerruf der Lifecycle-Credentials.
- **Backend ↔ Zielsystem:** Übergabe nach Signaturabschluss; die Bestätigung kommt asynchron als Callback des Zielsystems, das sich als sein eigener OAuth2-Client authentifiziert (Kapitel 04).
- **Öffentliche Flächen:** Ohne Authentifizierung erreichbar sind DID-Dokument, Agreement Credential samt Regelwerk, der C2PA-Manifest-Store und die Semantic-Hub-Auflösung (Anhang A).

Vertrauens- und Schlüsselarchitektur

Die Schlüssel im PKCS#11-Token sind nach Zweck getrennt: Instanz-DID, VC-Ausstellung, OID4VP-Request-Signatur, C2PA-Manifestsignatur und Schlüsselvereinbarung für die Artefaktverschlüsselung (Kapitel 05). pdf-core, Frontend und alle Fremdkomponenten sind gegenüber der Instanzidentität schlüssellos. Einen Vertrags-Signaturschlüssel hält die Instanz bewusst nicht; den hält der Signatar (Kapitel 06).

Die Vertrauensprüfung gegenüber Peers ruht auf unabhängigen, fail-closed arbeitenden Schichten: Zertifikatskette der Peer-DID, Challenge-Response-Signatur pro Request, das Trust Gate aus Agreement-Credential und lokalem Policy-Endpunkt (ADR-19) sowie die Prüfung der Handlungsvollmacht hinter jeder Signatur der Gegenseite (ADR-31, Kapitel 08).

Betriebsicht

Alle Komponenten einer Instanz werden über ein gemeinsames Helm-Chart deployt; zwei vollständige Instanzen lassen sich parallel betreiben. Erforderliche externe Abhängigkeiten prüft das Backend beim Start und schlägt hart fehl, statt degradiert zu laufen (Kapitel 10).

03 Vorlagen und Vertragsinhalte

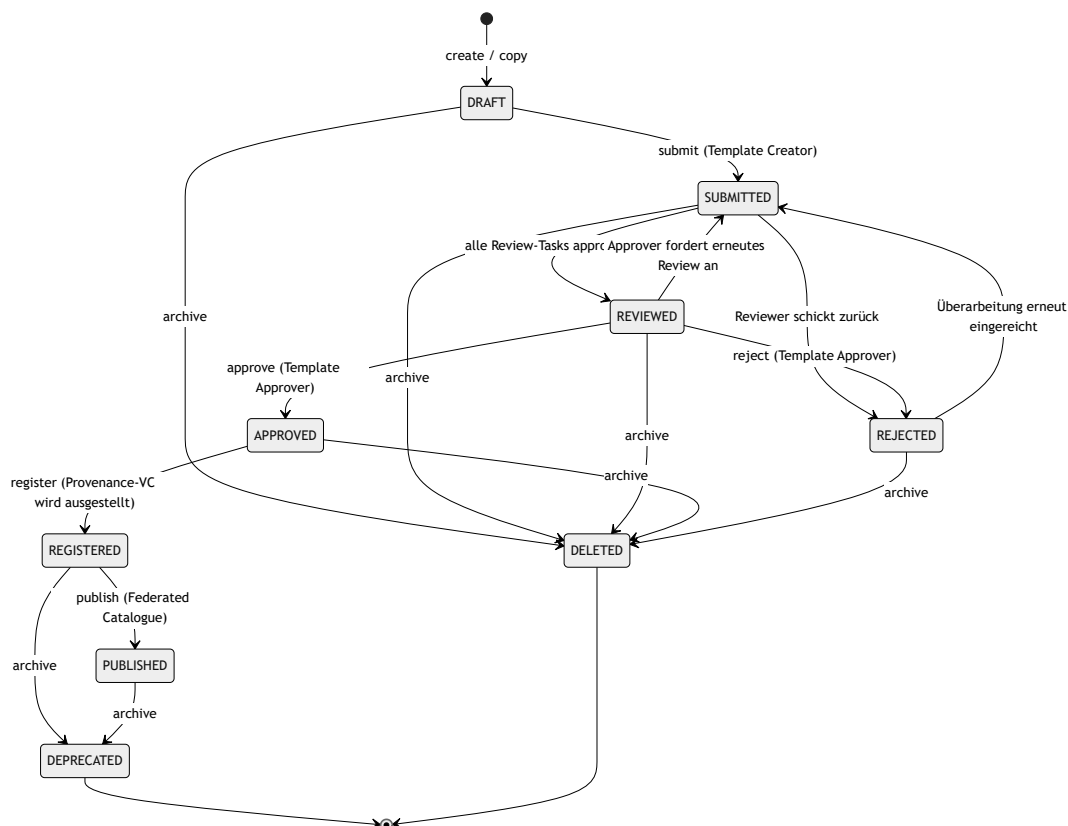
Dieses Kapitel beschreibt, wie Vertragsvorlagen entstehen, geprüft, versioniert und veröffentlicht werden, und wie der Semantic Hub die maschinenlesbare Bedeutung aller Inhalte definiert: JSON-LD-Kontext, SHACL-Shapes, Domänenfeld-Katalog, ODRL-Profil und Klausel-Katalog.

3.1 Beteiligte Komponenten

Komponente	Verantwortung
Template Repository	Vorlagen-Lifecycle, Review-/Approval-Tasks, Versionierung, Provenance
Semantic Hub	Versionierter Speicher und Auslieferungspunkt aller semantischen Artefakte
Federated Catalogue (XFSC FC)	Externer Katalog, in den registrierte Vorlagen als Self-Descriptions veröffentlicht werden
Template Catalogue Integration	Lesezugriff auf den Katalog; so beziehen andere Instanzen veröffentlichte Vorlagen
pdf-core	Deterministisches Rendering der Dokumente (Kapitel 07)

3.2 Template-Lifecycle

Eine Vorlage durchläuft einen mehrstufigen Freigabeprozess mit Vier-Augen-Prinzip, bevor sie für die Vertragserstellung nutzbar und im Federated Catalogue sichtbar wird.



Ergänzend zum Diagramm:

- Jede schreibende Operation prüft den mitgelieferten Zeitstempel gegen den Stand der Vorlage; ein veralteter Stand wird mit der Aufforderung zum Neuladen abgewiesen (Lost-Update-Schutz).
- Ein Reviewer muss die Vorlage erst semantisch verifizieren, bevor er sie freigeben kann. Rollenprüfung und Task-Zuständigkeit sind zwei getrennte Hürden: Die RBAC-Rolle erlaubt die Operation, der Task bindet sie an die konkrete Vorlage.
- Die Registrierung fixiert eine Version als die veröffentlichungsfähige; die Instanz stellt dabei ein Provenance-Verifiable-Credential über genau diese Version aus.
- Beim Publish tritt die DCS-Instanz (deren `did:web`) als Aussteller auf, nicht der klickende Mitarbeiter (ADR-18). Der Katalog-Aufruf läuft außerhalb der Datenbanktransaktion; ein vom Katalog gemeldetes Duplikat gilt als Erfolg, so dass ein wiederholter Publish nach einem Teilfehler idempotent ist.
- `archive` wirkt zustandsabhängig: registrierte oder veröffentlichte Vorlagen werden `DEPRECATED` (bleiben referenzierbar), alle früheren Zustände werden `DELETED`.

Die Lifecycle-Ereignisse erreichen registrierte externe Abonnenten über die Webhook-Plattform (Anhang A).

Versionierung durch Kopie. Vorlagen werden nicht in-place versioniert. Der Copy-Befehl dupliziert eine Vorlage unter neuer DID: Eine unregistrierte Quelle beginnt eine neue Versionslinie, eine registrierte oder veröffentlichte liefert die nächste Version derselben Linie. Jede neue Version durchläuft den vollen Freigabeprozess erneut.

Rolle	Darf
Template Creator	Vorlagen anlegen, bearbeiten, kopieren, einreichen
Template Reviewer	Eingereichte Vorlagen verifizieren und reviewen
Template Approver	Reviewte Vorlagen freigeben oder ablehnen
Template Manager	Übergreifende Verwaltungsrechte; einziger Rolleninhaber, der Semantic-Hub-Versionen registrieren darf

Vorlagen werden nicht direkt zwischen Instanzen synchronisiert; der Austauschweg ist die Veröffentlichung in den Katalog und der Lesezugriff anderer Instanzen über die Catalogue-Integration. Der Katalog übergibt dabei Inhalt, nie Autorität: Eine importierende Instanz registriert die gefundene Vorlage als eigene neue Vorlage unter eigener DID in `DRAFT` und führt sie durch ihren eigenen Freigabeprozess. Die ODRL-Semantik reist unverändert mit und bindet die importierende Instanz wie die Autorin: Dieselben Regeln, Bereichsgrenzen und Konsequenzen werden auf beiden Instanzen mit demselben Ergebnis durchgesetzt.

3.3 Semantic Hub

Der Semantic Hub ist der versionierte Speicher für alles, wogegen das DCS seine Dokumente erzeugt und prüft. Jeder Eintrag ist ein Paar aus Name und Art (`kind`) mit unveränderlicher Versionshistorie und genau einer aktiven Version. Die mit dem Backend ausgelieferten Basisdokumente:

Hub-Eintrag (Name / Art)	Inhalt und Wirkung
facis-dcs / context	Der JSON-LD-Kontext, den jedes Dokument als <code>@context</code> verankert; seine Präfix-zu-IRI-Zuordnungen werden auf jedem Artefakt erzwungen
facis-dcs / shapes	SHACL-Shapes der kanonischen Dokumenthülle, der blockierende Validierungsgraph
clause-catalog / shapes	Typisierte Klausel-NodeShapes; zugleich die Deklaration ihrer Begriffe (3.4)
facis-sla / ontology	Der Domänenfeld-Katalog: <code>dcs:DomainField</code> -Einträge mit Wertebereichen und Taxonomie-Optionen; geparte RDF-Konfiguration
dcs-odrl-profile / ontology	Das ODRL-Profil: Constraint-Operatoren, Aktionen, Default-Aktion. Treibt den Bedingungs-Editor
facis.sla.basic / profile	Fachliche Validierungsregeln auf Statement-Ebene
beliebige Shape-Bibliotheken / shapes	Über den Hub registrierte externe SHACL-Vokabulare (z. B. Gaia-X-Klassen): Bausteine für typisierte Geschäftsobjekte in <code>dcs:contractData</code>

Eine OWL-Ontologie der Dokumenthülle gibt es bewusst nicht: Maßgeblich sind die Shapes, gegen die validiert wird; im System läuft kein Reasoner.

Die Wirkmechanik:

- **Seed beim Start.** Die Basisdokumente werden beim Start installiert; ein vom letzten Stand abweichendes Dokument wird nächste aktive Version. Versionen sind unveränderlich. Schlägt der Seed fehl, startet die Instanz nicht.
- **Pinning statt Mitwandern (ADR-8).** Ein neu erzeugtes Dokument verankert die aktive Version über versionsfeste URLs. Eine spätere Aktivierung ändert nur, wogegen neue Dokumente validiert werden; bestehende bleiben an ihre Version gebunden und dauerhaft reproduzierbar prüfbar. Für Korrekturen existiert ein expliziter Rollback.
- **Öffentliche Auflösung.** Die Auflösungsendpunkte sind unauthentifiziert, damit externe Prüfer die im Dokument verankerten Anker ohne DCS-Konto dereferenzieren können.
- **Hermetische Auflösung.** Ein Dokument darf externe Kontext-IRIs nur referenzieren, wenn sie unter ihrer Original-IRI im Hub registriert sind; validiert wird ausschließlich gegen den Hub-Bestand, nie über das Netz. Eine unregistrierte IRI wird bei der Erstellung abgewiesen.

Was wo geprüft wird

Schicht	Gegenstand	Wirkung
SHACL gegen den gepinnten Shapes-Graphen	Struktur, Typen, Kardinalitäten der Hülle; Wertbedingungen des Klausel-Katalogs und aller aktiven Shape-Bibliotheken	Blockierend bei Angebot, Einreichung und Signaturvorbereitung
ODRL-Constraint-Auswertung	Die im Vertrag inline getragenen Werte gegen die eigenen ODRL-Bedingungen	Blockierend bei Freigabe und Signaturvorbereitung (Kapitel 09)
Abschluss-Prüfung	Offene Pflichtfelder und von ODRL-Regeln referenzierte, unbelegte Werte	Blockierend bei Angebot, Freigabe und Signaturvorbereitung
Validierungsprofil (Statement-Regeln)	Fachliche Regeln über Aussagen im Dokument	Fließt in das Workflow-Gate ein: „error“ blockiert, eine Warnung stellt unter manuelle Prüfung (Kapitel 09)
Wertbedingungen des Domänenfeld-Katalogs	Zulässige Werte eines Domänenfelds	Nicht serverseitig durchgesetzt ; sie treiben Editor und clientseitige Prüfung. Wer eine Wertgrenze erzwingen will, drückt sie als SHACL aus

Bei der Signaturvorbereitung wird SHACL-Evidenz (Shapes-Version und ein Hash über die Befundliste) in das Signing-Summary-Credential eingebettet, so dass externe Prüfer die Validierung gegen exakt die gepinnten Shapes wiederholen können.

3.4 Klausel-Katalog

Der Klausel-Katalog ist ein eigenständig versionierter Shapes-Eintrag im Hub: typisierte Klausel-NodeShapes mit Zielklassen, Datentypen, Wertelisten, Bereichsgrenzen und Mustern. Er bedient zwei Konsumenten aus einer Quelle: Der Klausel-Endpunkt liefert dem Vorlagen-Builder die Palette der Klauseltypen, die Formulare entstehen clientseitig aus dem rohen Turtle (ADR-10); eingereichte Klausel-Instanzen werden serverseitig gegen denselben Shapes-Graphen validiert. Palette und Prüfung können damit nicht auseinanderlaufen. Der Katalog ist zugleich die Deklaration seiner Begriffe; ein separates Vokabular existiert nicht.

3.5 Die kanonische Dokumenthülle und das Contract-Field-Modell

Jedes DCS-Dokument, Vorlage wie Vertrag, ist eine einzige kanonische JSON-LD-Hülle (`dc:ContractTemplate` bzw. `dc:Contract`):

Ebene	Inhalt
<code>dcs:metadata</code>	Titel, Beschreibung, Vorlagentyp
<code>dcs:documentStructure</code>	Menschenlesbare Struktur: geordnete Blöcke (<code>dcs:Section</code> , <code>dcs:TextBlock</code> , <code>dcs:Clause</code>) und ein Layout-Baum
<code>dcs:contractFields</code>	Flache Liste der Felddeklarationen (<code>dcs:ContractField</code>)
<code>dcs:contractData</code>	Generischer Graph typisierter Geschäftsobjekte; Eigenschaften tragen Literale, Feldreferenzen oder Objektreferenzen
<code>dcs:policies</code>	Genau eine umschließende ODRL-Policy
<code>dcs:signatureFields</code>	Die deklarierten Signaturfelder mit gefordertem Signaturniveau (Kapitel 06)

Das Contract-Field-Modell (ADR-22, erweitert durch ADR-23) trennt Felddeklaration und Geschäftsdaten: Jedes `dcs:ContractField` deklariert `@id`, Label, Datentyp und Pflichtkennzeichen. Im Datengraphen trägt eine Eigenschaft genau eines von dreien: ein Literal, eine Referenz auf ein deklariertes Feld (ein verhandelbares Blatt) oder eine Referenz auf ein anderes Geschäftsobjekt. Jede Referenz muss sich im Dokument selbst auflösen; eingebettete Blank Nodes sind unzulässig. Klausel-Prosa, Layout und ODRL-Operanden referenzieren dieselben Feld-IDs. Das Dokument ist dadurch selbsttragend: Authoring, Validierung, Rendering und Policy-Auswertung lösen ein Feld über seine `@id` auf, ohne einen Vorlagen-Schnappschuss zu konsultieren.

Offene Felder und das Abschluss-Gate. Auf einer Vorlage darf ein Feld deklariert, aber ohne Wert sein; gefüllt wird es bei Vertragserzeugung und Verhandlung. Angebot, Freigabe und Signaturvorbereitung weisen einen Vertrag mit ungefüllten Pflichtfeldern oder von ODRL-Regeln referenzierten, leeren Werten zurück. Keine Partei kann ein Dokument mit offenen Bedingungen signieren (Kapitel 04).

Hierarchie. Ein Vertrag darf auf genau einen Elternvertrag verweisen; die Verknüpfung zeigt ausschließlich vom Kind zum Elternteil (ADR-7). Rückwärtsverweise werden beim Normalisieren abgewiesen, weil sie auf Dokumente zeigen könnten, die der Leser nicht sehen darf.

Shape-Bibliotheken treiben Editor und Validierung. Registrierte SHACL-Bibliotheken treten dem aktiven Validierungsgraphen bei; der Editor leitet daraus seine Palette typisierter Datenobjekte ab. Validiert wird gegen eine feld-materialisierte Kopie des Dokuments: Referenzen auf gefüllte Felder werden zu ihrem Wert dereferenziert, so dass gewöhnliche, gegen Instanzdaten geschriebene Bibliotheken das Dokument direkt beschränken. Referenzen auf ungefüllte Felder fehlen in der Kopie; die Kardinalitätsregeln benennen dann, was die Verhandlung noch liefern muss (ADR-23).

ODRL-Regeln. Die Policy ist bis zur ersten Signatur ein `odr1:Offer` und wird durch die Signatur in ein `odr1:Agreement` gesiegelt. Jede Regel trägt Aktion, Assigner, Assignee, Ziel und die menschenlesbare Klausel, die sie operationalisiert; die unabhängige Gegenprüfung beider Repräsentationen beschreibt Kapitel 07.

Zur Laufzeit gibt es genau **eine** interne Form, die kanonische `dcs:`-präfixierte Hülle. Sie wird an den Eingangskanten erzwungen: Ein Dokument, dessen `@context` einen Hub-Präfix auf eine andere IRI umbiegt, wird abgewiesen. Ein von einer Peer-Instanz empfangenes Dokument kommt bereits in dieser Form an, weil das PDF genau diese Bytes als Anhang trägt (Kapitel 07). OWL-Inferenz findet nicht statt.

3.6 Betriebsverhalten

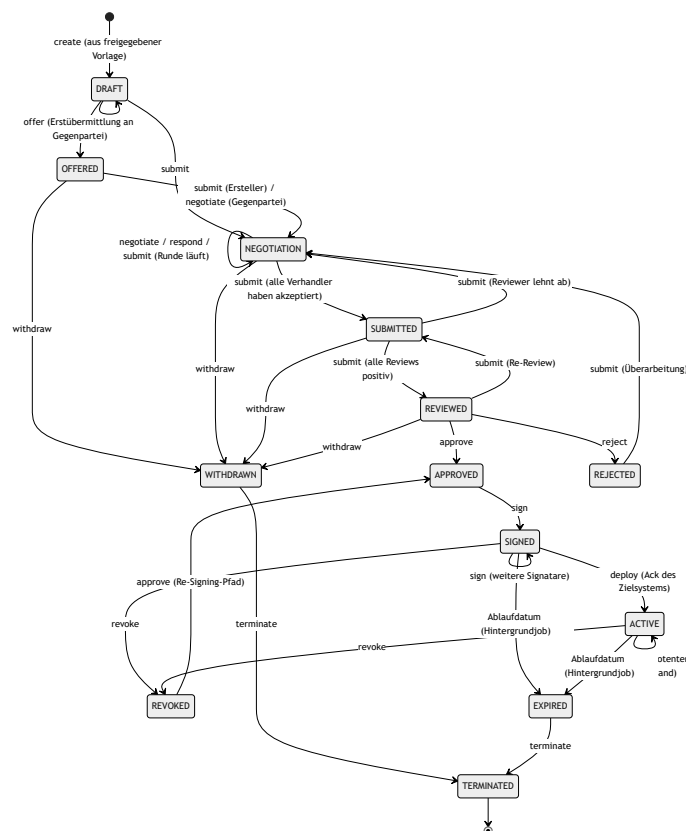
- Ohne konfigurierten Federated Catalogue schlägt `publish` mit einem eindeutigen Konfigurationsfehler fehl; alle vorgelagerten Schritte funktionieren unabhängig davon. Ein konfigurierter Katalog ist eine harte Startabhängigkeit (Kapitel 10).
- Schlägt nach erfolgreicher Katalog-Veröffentlichung die lokale Zustandsänderung fehl, heilt der nächste Publish-Aufruf den Zustand.
- Konkurrierende Bearbeitung wird über den Änderungszeitstempel erkannt und als Client-Fehler beantwortet; ein stiller Überschreib findet nicht statt.
- Jeder Lifecycle-Schritt erzeugt sein Audit-Ereignis in derselben Transaktion wie die Zustandsänderung (Kapitel 09).

04 Vertragslebenszyklus

Dieses Kapitel beschreibt den Lebenszyklus eines Vertrags von der Erstellung aus einer freigegebenen Vorlage über Verhandlung, Review und Freigabe bis zu Signatur, Aktivierung, Beendigung und Löschung: die zentrale State-Machine, das Berechtigungsmodell aus Rollen und Tasks, die Workflow-Gates, die Verhandlung sowie den Unterschied zwischen einem lokalen und einem instanzübergreifenden Vertrag.

4.1 Die State-Machine

Es gibt genau eine Vertrags-State-Machine im gesamten Backend (ADR-2). Zulässige Übergänge sind als explizite Tabelle hinterlegt; jeder Command-Handler validiert dagegen, bevor er seine Fachlogik ausführt. Ein unzulässiger Übergang ist ein Client-Fehler, kein interner Fehler.



Ergänzend zum Diagramm:

- **Terminate** ist aus jedem nicht-terminalen Zustand erreichbar (im Diagramm nur exemplarisch gezeigt).
- **Submit ist bewusst überladen:** Seine Wirkung hängt vom Zustand ab (4.4). Die Übergangstabelle begrenzt die legalen Ausgänge; welcher eintritt, entscheidet die Task-Orchestrierung.
- **Withdraw** ist nur bis zur Freigabe möglich.
- **Expired** setzt ein Hintergrundjob anhand des Ablaufdatums, aus jedem laufenden Zustand vom Angebot bis `ACTIVE` (im Diagramm exemplarisch ab `SIGNED`); Entwürfe sowie abgelehnte, zurückgezogene und widerrufenen Verträge lässt er unangetastet. Eine Datenbanksicht zeigt den Zustand schon vorher zum Abfragezeitpunkt. Mit dem Übergang wird `contract.expired` publiziert.

- **Renew** ist kein Zustandsübergang: Die Verlängerung erzeugt eine neue, verknüpfte Vertragsinstanz in **DRAFT** mit Rückreferenz auf das Original. Die Signaturen der Vorperiode werden aus dem neuen Entwurf entfernt.

Extrinsischer Lebenszyklus. Über die Instanzgrenze präsentiert jeder Vertrag einen abgeleiteten Verhandlungslebenszyklus: alle Formationszustände bis **REVIEWED** als **proposed**, die beidseitige interne Freigabe als **agreed** (hier öffnet sich das Signatur-Gate), ein Vertrag mit vollständig vorliegenden Signaturen als **executed**. Für **executed** muss jede deklarierte Signatur tatsächlich vorliegen, einschließlich der kryptographisch geprüften Signatur der Gegenseite. Beide Instanzen leiten denselben Wert aus dem gemeinsamen Artefakt ab; es wird kein Zustand über die Föderationsgrenze synchronisiert.

4.2 Rollen, Tasks und RBAC

Die Autorisierung hat zwei getrennte Schichten:

1. **Lokale RBAC-Rollen** bestimmen, welcher Nutzer eine Operation grundsätzlich ausführen darf: Contract Creator, Reviewer, Approver, Negotiator, Manager, Signer und Observer. Maschinelle Aufrufer treten in einer eigenen **sys.**-Klasse auf, die nur einen Teil davon spiegelt; für Verhandeln und Beobachten gibt es kein maschinelles Pendant, und die maschinelle Signaturklasse trägt keine Signaturberechtigung (Kapitel 05 und 06). Audit-Lesezugriff haben Auditor, Archive Manager und Compliance Officer.
2. **Tasks binden Zuständigkeit an den konkreten Vertrag.** Die Rolle erlaubt die Operation, der Task entscheidet, ob dieser Aufrufer für diesen Vertrag zuständig ist.

Die Tasks sind kleine Sub-State-Machines, die als Fan-in-Gates die großen Übergänge steuern:

Task	Zustände	Gate
Negotiation-Task	offen → akzeptiert	Erst wenn kein Verhandler-Task mehr offen ist, kann NEGOTIATION → SUBMITTED erfolgen
Review-Task	offen → freigegeben / abgelehnt	Steuert SUBMITTED → REVIEWED ; eine Ablehnung öffnet die Verhandlung erneut
Approval-Task	offen → freigegeben / abgelehnt	Steuert REVIEWED → APPROVED

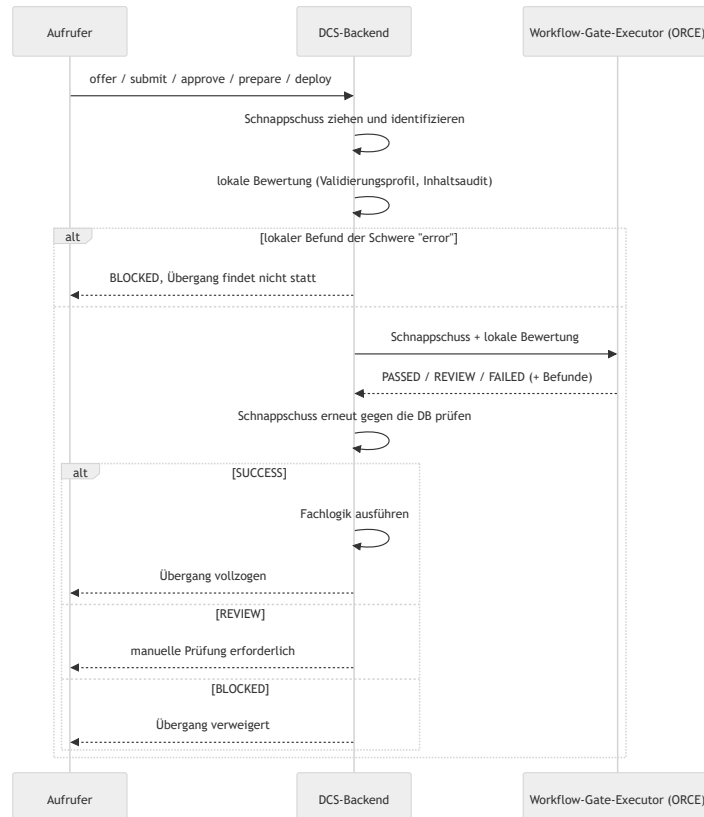
Jede Instanz erstellt und besitzt ausschließlich ihre eigenen Tasks; Task-Zustände überqueren nie die Instanzgrenze (4.5).

Der Lebenszyklus ist API-vollständig. Jeder Schritt von der Erstellung über Review und Freigabe bis zu Signatur und Archivierung ist einzeln über die authentifizierte HTTP-API auslösbar und erzeugt dieselben Zustands- und Evidenzketten wie die Bedienung über die Oberfläche; das Frontend ist nur ein weiterer API-Client. Maschinelle Aufrufer führen Verträge damit vollständig systemgesteuert (Kapitel 05). Allein die Signatur bleibt eine persönliche Wallet-Ceremony, die ebenfalls ohne Oberfläche über die API läuft (Kapitel 06).

Für Lesezugriffe gilt zusätzlich eine Parteiprüfung: Das kanonische Vertragsdokument ist nur für autorisierte Parteien lesbar. Eine Verweigerung wird als Audit-Artefakt festgehalten, bevor die Ablehnung zurückgeht; daraus speist sich das Compliance-Risiko „unautorisierter Zugriff“ (Kapitel 09).

4.3 Workflow-Gates: der auslagerbare Prüfschritt vor dem Übergang

Fünf Übergänge laufen zuerst durch ein **Workflow-Gate**: Angebot, Einreichung, Freigabe, Signaturvorbereitung und Deployment. Ein Gate arbeitet auf einem unveränderlichen Schnappschuss des Vertrags (Version, Zustand, Inhalts-Hash, gepinnte Shapes, Profilversion):



- **Der Lauf ist Evidenz.** Jeder Gate-Lauf wird mit Korrelations-ID, Schnappschuss-Identität, Anfrage und Antwort persistiert und ist über die Compliance-API abrufbar. Zwei Läufe desselben Schnappschusses am selben Gate sind derselbe Lauf.
- **Der Schnappschuss wird nach der Antwort erneut geprüft.** Eine zwischenzeitliche inhaltliche Änderung verwirft das Ergebnis; rein technische Schreibvorgänge (PDF-Regeneration, Audit) blockieren nicht.
- **REVIEW ist eine Wartestellung, kein Fehler.** Ein Compliance Officer entscheidet den Lauf mit Begründung; bei Freigabe wird genau der angehaltene Übergang fortgesetzt.
- **Fail-closed.** Ein nicht konfigurierter oder nicht erreichbarer Executor blockiert; der Lauf wird trotzdem mit Ursache festgehalten. Der Executor ist deshalb eine harte Startabhängigkeit.

4.4 Verhandlung

Die Verhandlung findet in `NEGOTIATION` statt, oder auf einem eingegangenen Angebot in `OFFERED`, und dreht sich um Change-Requests:

- **Vorschlagen.** Ein Change-Request ist entweder Freitext (eine Anmerkung nur für den Verhandlungs-Audit-Trail) oder ein strukturierter Redline auf die Vertragsdaten. Ein Redline wird sofort auf die Vertragsdaten angewendet; das PDF rendert mit dem vorgeschlagenen Wert neu und

wird an die Gegenpartei verschifft (4.5), die den Redline im empfangenen Dokument begutachtet.

- **Entscheiden.** Jeder zuständige Verhandler akzeptiert oder lehnt einen Change-Request ab (Ablehnung mit Begründung).
- **Runde abschließen.** Submit ist der Rundenabschluss: Sind noch Verhandler-Tasks offen, bleibt der Vertrag in `NEGOTIATION`; sind akzeptierte Change-Requests zu mergen, werden sie zusammengeführt, die Version zählt hoch, und eine neue Runde beginnt; ist nichts mehr zu mergen, wechselt der Vertrag nach `SUBMITTED`.

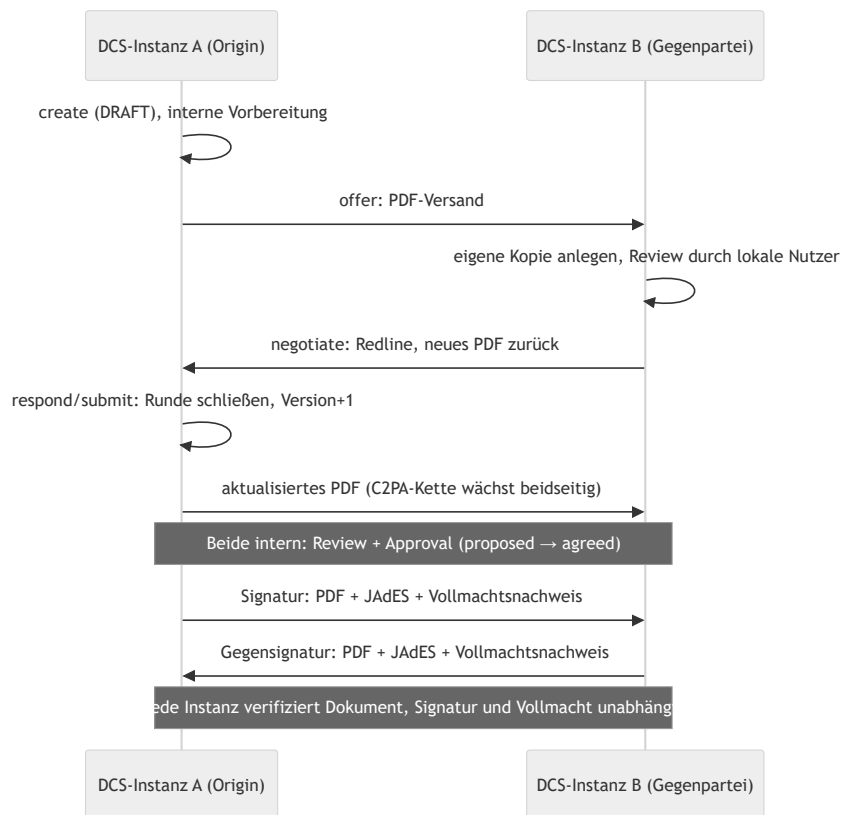
Verhandlungs-Drafts. Ein Verhandler kann einen Change-Request vorbereiten: Der Draft wird pro (Vertrag, Partei) gespeichert und von allen Verhaltern derselben Partei geteilt. Er ändert keinen Zustand, erzeugt kein Audit-Ereignis und verlässt die Instanz nicht; erst das Vorschlagen macht ihn zum Change-Request und löscht ihn.

4.5 Lokal vs. grenzüberschreitend

Ein Vertrag hat genau eine **Ursprungsinstanz** (Origin) und genau eine **Gegenpartei**, ein Peer-DCS, identifiziert über `did:web` und bei der Vertragserstellung benannt. Lokal (Origin gleich aufrufende Instanz) sind alle Zuständigkeiten lokale Tasks, und nichts verlässt die Instanz außer einem optionalen Deployment.

Grenzüberschreitend führt **jede Instanz die State-Machine auf ihrer eigenen Kopie**; Aufrufe werden nicht an die Origin weitergereicht. Über die Grenze wandert ausschließlich das Vertragsdokument (ADR-13). Die Autorisierung spiegelt das: Auf der Origin-Instanz verlangt das Verhandeln einen lokalen Verhandler-Task; auf der Empfängerseite leitet sich das Recht aus der Eigenschaft als benannte Gegenpartei ab. Das Bearbeiten des Entwurfs (update) ist nur auf der Origin-Instanz zulässig.

Das PDF ist das Wire-Format. Bei jedem versandpflichtigen Schritt wird das Vertrags-PDF verschifft: mit eingebettetem JSON-LD, C2PA-Provenance-Kette und vorhandenen Signaturen. Ein nacktes PDF ist ein Vorschlag; ein PDF mit beigefügter JAdES ist die Signatur der Gegenseite. Der Empfänger authentifiziert den Absender über eine did:web-Challenge-Response-Signatur, lässt pdf-core prüfen, dass der Seiteninhalt der Re-Render der eingebetteten Payload ist, und aktualisiert seine lokale Kopie (Kapitel 07 und 08).



Trust-Modell und Peer-Authentifizierung behandelt Kapitel 08, die Signatur-Zeremonie Kapitel 06.

4.6 Nach der Signatur: Deployment, Widerruf, Beendigung

Zielsystem-Registry und Designation (ADR-25)

Contract Target Systems sind eine administrierte Registry der Instanz mit Name, Endpunkt und Aktiv-Kennzeichen. Jeder Vertrag **designiert** sein Deployment-Ziel. Ein deaktiviertes Ziel kann nicht designiert werden; ein designiertes Ziel kann nicht gelöscht werden. Ein Vertrag ohne designiertes Ziel deployt nicht, und das ist ein normaler Ausgang: Der automatische Trigger nach Signaturabschluss tut dann nichts, ein manueller Deploy wird mit Begründung abgewiesen.

Deployment

Ein vollständig signierter Vertrag wird als JSON-LD-Payload mit Vertrags-DID, Version, Content-Hash, Zeitstempel und der ODRL-Policy an das designierte Ziel übermittelt; ein manueller Deploy darf per Override an ein anderes registriertes Ziel gehen, ohne die Designation zu ändern.

Das **Deploy-Gate** verlangt, dass jedes deklarierte Signaturfeld signiert ist. Für die eigenen Felder zählt die lokal aufgezeichnete Signatur, für die Gegenseite das verifizierte JAdES-Artefakt aus deren signierter Sendung. Eine Partei kann einen Vertrag nicht allein in die Ausführung bringen. Die Instanz hält je Vertrag genau eine Gegenseiten-Signatur; ein Vertrag mit drei oder mehr Parteien wird deshalb nicht deployt, statt eine unvollständige Signaturlage als vollständig zu behandeln.

Der Deployment-Datensatz hält den Endpunkt fest, wie er beim Versand galt; der Versand trägt eine Correlation-ID und einen nachrechenbaren Content-Hash. Ein fehlgeschlagener Versand steht am Datensatz und wird vom Compliance-Monitor als Risiko gemeldet. Die Bestätigung kommt asynchron als Callback und schaltet den Vertrag auf `ACTIVE`; das Zielsystem authentifiziert sich als sein eigener OAuth2-Client und kann nur eigene Deployments quittieren (ADR-27). Die quittierte Ausführungs-Evidenz erhält einen RFC-3161-Zeitstempel und steht am Archiv-Eintrag. Über denselben Callback meldet das Zielsystem später KPI-Beobachtungen samt seinem Urteil zur jeweiligen ODRL-Regel (Kapitel 09).

Widerruf, Beendigung, Ablauf

- **Widerruf** versetzt den Vertrag von `SIGNED / ACTIVE` nach `REVOKED`; ein erneutes Approve führt zurück nach `APPROVED` (Re-Signing). Bei einem grenzüberschreitenden Vertrag geht der Widerruf unmittelbar an die Gegenpartei und ist der einzige Peer-Zustand, den der Empfänger übernimmt (Kapitel 08).
- **Terminate** mit Begründung beendet den Vertrag endgültig.
- **Ablauf:** Der Hintergrundjob persistiert `EXPIRED` und publiziert das Ereignis. Die hinterlegte Expiration-Policy wird dabei **aufgezeichnet, aber nicht automatisch ausgeführt**; die Folgehandlung ist manuell oder über Webhooks orchestriert. Ein Vertrag ohne Expiration-Policy wird vom Job übersprungen und protokolliert.

Archiv und Löschung

Abgeschlossene Verträge liegen mit ihrer Evidenz im Langzeitarchiv, strukturiert durchsuchbar (Volltext, Name, Zustand, Partei, Zeitraum, Tags), mit Dashboard für Bestands- und Compliance-Statistiken.

Die Löschung eines Archiveintrags ist begründungspflichtig und hat zwei Wirkungen: Der Eintrag wird als gelöscht markiert und bleibt nachweisbar, die archivierten Snapshots werden entfernt; und die **Inhaltsschlüssel des Vertrags werden vernichtet** (ADR-28), lokal und per Aufforderung an jede Gegenpartei-Instanz. Danach sind Export, Verifikation und Bundle-Abruf dauerhaft nicht mehr möglich; Listen- und Metadatensichten funktionieren weiter. Eine nicht erreichbare Gegenseite blockiert nicht: Die Anforderung bleibt offen und wird periodisch erneut zugestellt. Der Fortschritt ist über eine eigene Statusabfrage sichtbar (Kapitel 09).

4.7 Betriebsverhalten

- **Optimistische Nebenläufigkeit:** Zustandsverändernde Endpunkte verlangen den Zeitstempel des gesehenen Standes; ein veralteter Stand wird mit „bitte neu laden“ abgewiesen, bei synchronisierten Verträgen mit „Synchronisation erzwingen und neu laden“. Verglichen wird gegen den Inhalts-Zeitstempel, so dass Hintergrundschreibvorgänge keine Fehlalarme auslösen.
- **Unzulässige Übergänge** fängt die Übergangstabelle als Client-Fehler mit sprechender Meldung ab.
- **Blockierende Prüfungen:** SHACL-Fehlerbefunde und offene Pflichtfelder oder ODRL-referenzierte leere Werte verhindern Angebot, Freigabe und Signaturvorbereitung (Kapitel 03).
- **Jede Zustandsänderung** schreibt ihr Audit-Ereignis in derselben Transaktion; der Audit-Trail ist lückenlos gegenüber der Zustandshistorie (Kapitel 09).
- **Lebenszyklus-Ereignisse** erreichen externe Abonnenten über die Webhook-Plattform mit Zustellprotokoll und Quittung (Anhang A).

- **Deployment-Vorgänge** sind über Correlation-IDs mit ihren Callbacks korrelierbar; ein nie zugestellter Versand ist von einem quitierten unterscheidbar.

05 Identität und Authentifizierung

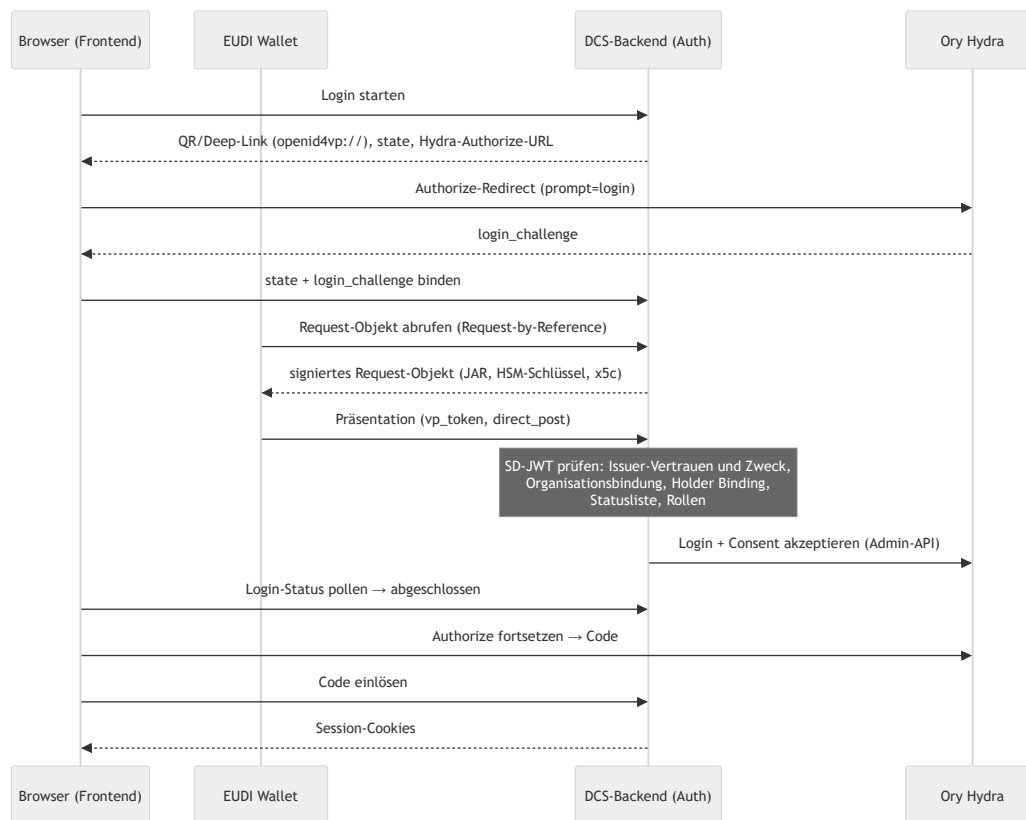
Das DCS kennt drei strikt getrennte Identitätsebenen: **Endnutzer** (Menschen mit Wallet), **Maschinen-Identitäten** (automatisierte Aufrufer mit eigenem OAuth2-Client) und die **DCS-Instanz selbst** (eine `did:web`-Identität mit HSM-gestütztem Schlüsselmaterial). Quer darüber liegt die Frage, wem die Instanz als Aussteller von Credentials glaubt, und wofür (5.4).

5.1 Endnutzer: Login per OID4VP und SD-JWT VC

Komponente	Rolle
EUDI Wallet (bzw. kompatible Wallet)	Hält die Credentials des Nutzers und legt sie per OpenID4VP vor
DCS-Backend (Auth-Domäne)	OID4VP-Verifier: baut das Presentation Request, prüft die Vorlage, entscheidet über den Login
Ory Hydra	OAuth2/OIDC-Provider: stellt die Tokens aus, verwaltet die Browser-Session
Frontend	Zeigt QR-Code / Deep Link, pollt den Login-Status, führt durch den OIDC-Flow

Hydra kennt keine Nutzer und keine Passwörter. Die Authentifizierungsentscheidung trifft das DCS-Backend anhand der verifizierten Wallet-Vorlage; Hydra ist die Token-Fabrik, deren Login- und Consent-Challenges das Backend über die Admin-API akzeptiert und dabei die verifizierten Claims (Subjekt, Organisation, Rollen) in die Session übernimmt.

Das Credential: Proof of Authority (PoA). Der Login verlangt ein Credential vom Typ `urn:dc:poa:v1` im Format `dc+sd-jwt` mit zwei entscheidenden Claims: `organization`, die Partei, für die der Nutzer handelt, und `roles`, die ihm eingeräumten Rollen. Die angefragten Credentials beschreibt eine DCQL-Query; die Standard-Query verlangt Holder Binding und ist per Konfiguration überschreibbar. Die `organization` ist eine Parteikennung, nicht die Identität des Deployments: Eine Instanz bedient legitimerweise mehrere Organisationen.



Das Request-Objekt (JAR) wird mit einem eigenen HSM-Schlüssel signiert und trägt die Zertifikatskette der Instanz im Header; der der Wallet präsentierte Client-Identifizierer folgt dem Schema `x509_san_dns` und entspricht dem SAN des Leaf-Zertifikats. Login-States sind zeitlich begrenzt und verlängerbar.

Verifikation der Vorlage. Jede Stufe ist ein hartes Abbruchkriterium:

1. **Aussteller und Zweck:** Der Aussteller muss in der Trust-Konfiguration stehen und für `login` zugelassen sein; sein Schlüssel wird über den für ihn deklarierten Mechanismus aufgelöst (5.4). Der Credential-Typ muss zugelassen sein.
2. **Organisationsbindung:** Die offengelegte `organization` muss zu den Organisationen gehören, für die dieser Aussteller sprechen darf.
3. **Holder Binding:** Subjekt und Key-Binding-JWT müssen an den Bindungsschlüssel des Credentials sowie an Audience, Nonce und Offenlegungs-Hash der laufenden Präsentation gebunden sein (Replay-Schutz).
4. **Statusliste:** Widerrufsprüfung auf jedem Pfad, für jeden Zweck; auch die Statusliste selbst muss signiert und vertrauenswürdig sein. Eine nicht erreichbare Statusliste führt zur Ablehnung.
5. **Rollen:** Jede offengelegte Rolle muss eine gültige DCS-Rolle sein; ohne Rollen kein Login. Die Rollen landen im Access-Token und sind die Grundlage der RBAC-Prüfung (Kapitel 04).

PID-Präsentation. Daneben gibt es einen sessionlosen PID-Präsentationsfluss für den Nachweis der natürlichen Person, insbesondere als Vorstufe der Signatur-Ceremony (Kapitel 06). Die Prüfkette ist dieselbe mit Zweck `pid`; die Organisationsbindung entfällt. Ein PID-Aussteller ist konstruktionsbedingt ein Dritter: Die Instanz darf den Identitätsnachweis nicht selbst ausstellen, den sie später als Beweis akzeptiert (ADR-31).

Schutzmechanismen:

- **IP-Lockout:** Nach fünf fehlgeschlagenen Token-Validierungen im Zeitfenster wird die Quell-IP für 15 Minuten gesperrt. Eine gültige Anmeldung mit fehlender Rolle zählt nicht zum Lockout; sie wird als authentifiziertes Zugriffsereignis auditiert.
- **Audit-Trail:** Zugriffs- und Präsentationsversuche werden persistiert und in den Audit-Trail eingespeist (Kapitel 09).
- **Anfragebudget je Credential:** Oberhalb eines Ein-Minuten-Budgets antwortet das DCS mit `429` und `Retry-After`. Unauthentifizierte Routen zählen nicht; gezählt wird über einen gekürzten Hash des Credentials, nie über das rohe Token.

5.2 Maschinen-Identitäten: ausgestellte Credentials über Hydra

Maschinelle Aufrufer authentifizieren sich mit dem Client-Credentials-Grant bei Hydra. Jeder Aufrufer erhält einen **eigenen OAuth2-Client**, den das DCS über die Hydra-Admin-API provisioniert (ADR-27). Eine **Machine-Identity-Registry** verzeichnet je Identität den Client, die Partei, der die Aktionen zugerechnet werden, die Rollen und das Aktiv-Kennzeichen. Berechtigungen werden bei jeder Anfrage anhand der Client-ID aus der Registry gelesen, nie aus Token-Claims; ohne aktiven Registry-Eintrag wird kein maschineller Aufrufer akzeptiert.

Die Credentials sind **ausgestellt, nicht konfiguriert**: Das Client-Secret erscheint genau einmal in der Antwort; das DCS speichert es nie, Hydra hält nur einen Hash. Rotation invalidiert das alte Secret sofort, Löschen entfernt auch den Client, Deaktivieren wirkt bei der nächsten Anfrage; verwaltet wird das als auditiertes Betriebshandeln durch den Sys. Administrator. Die Deployment-Konfiguration `DCS_SYSTEM_CLIENTS` ist ein **Seed** für Aufrufer, die vor dem ersten Login existieren müssen; zur Laufzeit wird ausschließlich die Registry gelesen, und eine ungültige Rolle im Seed lässt den Start scheitern.

Contract Target Systems sind keine allgemeinen Maschinen-Identitäten: Ihre Rolle autorisiert ausschließlich den Deployment-Callback, und dort nur Deployments an genau dieses Ziel. Das Callback-Credential folgt demselben Mechanismus (Kapitel 04).

Rollengrenze Mensch/Maschine. Die `sys.`-Rollen sind bewusst nicht deckungsgleich mit den menschlichen: kein maschinelles Verhandeln oder Beobachten, keine Signaturberechtigung (maschinelles Signieren ist keine AES einer Person, ADR-17), Katalogzugriff und die Registrierung neuer Semantic-Hub-Versionen bleiben menschlichen Rollen vorbehalten.

Der clusterinterne Dienst-Token. Für den ereignisgetriebenen PDF-Regenerator existiert ein im Deployment gesetztes Dienst-Credential. Sein Abgleich findet **vor** Token-Validierung, Rollenprüfung und Lockout statt: Wer den Wert besitzt, ist auf jeder authentifizierten Route ein Aufrufer mit vollen Rechten. Der Wert ist entsprechend zu behandeln; das Chart erzwingt, dass ein Betreiber ihn setzt, einen Vorgabewert gibt es nicht. Herkunft und Ablage beschreibt der Deployment-Leitfaden.

5.3 Instanz-Identität: did:web mit HSM-Schlüsseln

Jede Instanz besitzt eine `did:web`-Identität; das DID-Dokument wird unter `/well-known/did.json` ausgeliefert. Die Auflösung folgt der `did:web`-Methode vollständig, auch für Identifier mit Pfadsegmenten, so dass mehrere Instanzen einen Host teilen können (Kapitel 08).

Die privaten Schlüssel liegen **ausschließlich** im PKCS#11-Token und verlassen es nie (ADR-1). Einen Software-Fallback gibt es nicht: Bei falschem Modulpfad, Token-Label oder PIN wird der Prozess nicht gesund. Ein fehlendes Token ist davon unterschieden: Der Prozess wartet darauf, weil bei einer frischen Installation die Provisionierung noch laufen kann (Kapitel 10).

Die Schlüssel sind nach Zweck getrennt (alle ECDSA P-256):

Standard-Label	Zweck
<code>dcS-did</code>	Instanz-Identität: JAdES-Signaturen der Föderation, DID-Challenge-Response
<code>dcS-vc</code>	Lifecycle-, Provenance- und Federation-Agreement-Credentials
<code>dcS-oid4vp-jar</code>	Signieren der OID4VP-Request-Objekte
<code>dcS-c2pa</code>	COSE-Signaturen der C2PA-Manifeste (Kapitel 07)
<code>dcS-ecdh</code>	Schlüsselvereinbarung: entpackt die Inhaltsschlüssel der Artefaktverschlüsselung (ADR-28)

Der Schlüsselvereinbarungs-Schlüssel ist eine harte Startabhängigkeit: Beim Start wickelt die Instanz einen Testschlüssel gegen den im DID-Dokument publizierten `keyAgreement`-Eintrag und packt ihn mit dem HSM wieder aus. Schlägt das fehl, wäre kein gespeichertes Artefakt lesbar, und die Instanz startet nicht.

Einen Vertrags-Signaturschlüssel hält die Instanz bewusst nicht: Vertragssignaturen erzeugt ausschließlich der Signatar mit dem eigenen Schlüssel (ADR-12/ADR-20, Kapitel 06). pdf-core hält nie einen Signer; es liefert nur zu signierende Daten zurück.

Schlüsselrotation erfolgt über versionierte Labels; alte Versionen bleiben im Token, damit historische Signaturen verifizierbar bleiben. Das Schlüsselinventar ist für den Sys. Administrator abfragbar; die Rotation selbst ist ein Betriebsverfahren (Key-Management-Konzept, Deployment-Leitfaden).

5.4 Ausstellervertrauen: zweckgebunden, organisationsgebunden, mechanismusoffen

Ob ein Credential akzeptiert wird, entscheiden drei getrennte Fragen (ADR-31):

Zweck. Jeder Aussteller wird für eine explizite Menge von Zwecken zugelassen; ein Eintrag ohne Zweckangabe wird beim Laden abgewiesen. Welche Aussteller welchen Zweck erhalten, ist Betreiberentscheidung.

Zweck	Bedeutung
<code>login</code>	Seine Credentials dürfen eine Session auf dieser Instanz begründen
<code>peer</code>	Seine Credentials werden in einer Signatur-Ceremony akzeptiert, und seine Aussage über die Vollmacht einer Gegenpartei wird geglaubt
<code>pid</code>	Er darf die Identität einer natürlichen Person attestieren

Organisation. Ein Aussteller darf nur Organisationen attestieren, die in seinem eigenen Eintrag stehen; eine fehlende Liste bedeutet niemals „beliebig“. PID-Aussteller sind ausgenommen, weil ein Identitätsnachweis eine Person attestiert und keine Organisation.

Mechanismus. Jeder Aussteller deklariert, wie sein Verifikationsschlüssel aufgelöst wird:

Mechanismus	Auflösung
jwks	Im Eintrag hinterlegte Schlüssel, über die Schlüssel-ID zugeordnet
x5c	Zertifikatskette im Credential-Header, verifiziert gegen konfigurierte Vertrauensanker
did:jwk	Schlüssel aus dem Ausstellerbezeichner dekodiert
did:web	Schlüssel aus dem DID-Dokument des Ausstellers
orcb	An einen konfigurierten ORCB-Flow delegiert

Ein nicht auflösbarer Mechanismus wird **beim Laden** abgewiesen: Ein Deployment erfährt von einer nicht unterstützten Trust-Konfiguration beim Start, nicht wenn die erste Wallet ankommt. Der `x5c`-Pfad ist streng: Ohne konfigurierte Vertrauensanker wird ein Credential mit Zertifikatskette abgewiesen, und das Leaf-Zertifikat muss den Aussteller benennen und für diesen Zweck ausgestellt sein, damit ein gewöhnliches TLS-Zertifikat desselben Hosts keine Credentials signieren kann.

Das Vertrauen ankert in dieser lokalen Konfiguration. Ob ein zugelassener Aussteller seinerseits durch eine übergeordnete Stelle legitimiert ist, etwa über eine Kettenprüfung bis zu einem Ökosystem-Anker, prüft das DCS nicht; diese Legitimationskette ist bewusst zurückgestellt.

Über der Trust-Konfiguration liegt eine **Autorisierungs-Policy** als eigenes Artefakt. Eine Policy, die sich nicht übersetzen lässt, stoppt den Start; eine, die sich zur Laufzeit nicht auswerten lässt, verweigert. Sie rangiert über dem Trust-Dokument, weshalb beide von der Startup-Attestierung erfasst werden (Kapitel 09).

5.5 Trust-Modell der Föderation im Überblick

Ob eine fremde Instanz als Peer akzeptiert wird, entscheidet das **Federation Trust Gate** (ADR-19), konsultiert bei Versand und Empfang: das Agreement-Credential des Peers (selbstsigniert, publiziert unter `/.well-known/dcs-agreement-credential.json`, mit dem Hash des einkompilierten Föderationsregelwerks) plus der lokale Policy-Decision-Point (`DCS_TRUST_PDP_URL`) als alleinige Autorisierungsinstanz. Das Gate arbeitet fail-closed; jede Ablehnung wird als Incident festgehalten. Den vollständigen Ablauf beschreibt Kapitel 08.

06 Signaturen

Dieses Kapitel beschreibt, wie ein Vertrag elektronisch signiert wird: die wallet-getriebene Signatur-Ceremony, die Signaturniveaus, die Formate PAdES (mensenlesbares PDF) und JAdES (maschinenlesbares JSON-LD), die Validierung über den EU Digital Signature Service (DSS) und die Zeitstempelung.

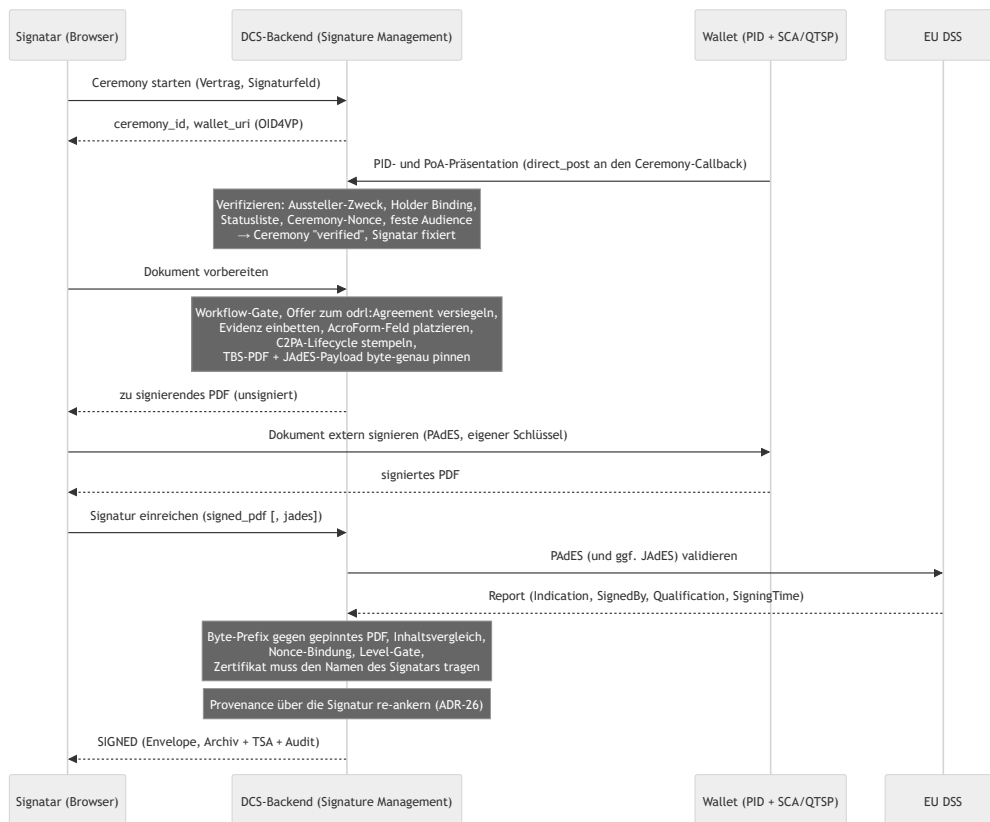
Das tragende Prinzip (ADR-12, ADR-3, ADR-20): **Das DCS signiert Verträge nie selbst und hält keinen Vertrags-Signaturschlüssel.** Es bereitet das Dokument vor, der Signatar signiert extern mit dem eigenen Schlüssel, und das DCS validiert und finalisiert das Ergebnis. Nur so ist die alleinige Kontrolle des Signatars über seinen Schlüssel (eIDAS Art. 26) erfüllbar. Was das DCS akzeptiert, muss byte-genau auf dem selbst vorbereiteten Dokument aufbauen, an die Ceremony gebunden sein und ein Zertifikat tragen, das den verifizierten Signatar benennt.

6.1 Beteiligte Komponenten

Komponente	Rolle
Signature Management (Backend)	Ceremony-Verwaltung, Dokumentvorbereitung, Annahme und Validierung externer Signaturen, Compliance-Sicht
Wallet + SCA/QTSP des Signatars	Legt das Identitätscredential vor, holt das Dokument, erzeugt die PAdES-Signatur mit dem Schlüssel des Signatars
pdf-core	Deterministisches Rendering des Basis-PDF, Inhaltsvergleich, Provenance-Re-Anchor; hält nie einen Signer (Kapitel 07)
EU DSS	Externer AdES-Validator (ETSI EN 319 102-1) für eingereichte PAdES- und JAdES-Signaturen
ORCE + TSA	RFC-3161-Zeitstempel für den Archiv-Eintrag; das Backend verifiziert die Antwort gegen das hinterlegte TSA-Zertifikat
Frontend (Signierliste, Secure Contract Viewer, Signature Compliance Viewer)	Führt den Signatar durch die Ceremony; zeigt Signaturstatus, Integritäts- und DSS-Befunde

6.2 Die Signatur-Ceremony

Signieren darf nur, wer zuvor in einer **Ceremony** seine Identität als natürliche Person nachgewiesen hat: eine OID4VP-Präsentation, gebunden an den Vertrag und ein deklariertes Signaturfeld. Ohne abgeschlossene Ceremony lehnt das DCS jede Dokumentvorbereitung und Signaturannahme ab.



Ergänzend zum Diagramm:

- Der Ceremony-Status ist pollbar (`pending` , `verified` , `expired` , `failed`); der Callback ist über die nicht erratbare Ceremony-ID autorisiert, und eine Präsentation an einer abgelaufenen Ceremony wird abgelehnt. Organisation und Rollen des Vollmachts-Credentials werden als Signaturevidenz aufbewahrt, weil die Gegenseite sie später nachprüft (Kapitel 08).
- Die Vorbereitung bettet die Signaturevidenz **innerhalb** des später signierten Byte-Bereichs ein (embed-then-sign, ADR-3) und **pinnt** die exakten zu signierenden Bytes (TBS-PDF und kanonische JAdES-Payload) an der Ceremony (ADR-20). Alle Seiteneffekte passieren genau einmal, hier.
- Die Einreichung ist ein reiner Validierungs- und Aufzeichnungsschritt (6.4). Die Finalisierung re-ankert die Provenance, erzeugt den Archiv-Eintrag mit Notarisierung und Zeitstempel und markiert den Verbrauch der Ceremony atomar, so dass zwei konkurrierende Einreichungen nicht beide finalisieren können.

Signatur und Provenance-Re-Anchor (ADR-26). C2PA-Hard-Binding und PAdES-Signatur wollen beide „das Letzte im Dokument“ sein. Das DCS löst den Konflikt in fester Reihenfolge: Lifecycle-Manifest vor der Signatur (die Signatur deckt die Provenance ab), nach der validierten Einreichung ein reines Provenance-Update-Manifest als inkrementelles Update über den signierten Bytes. Die Signatur bleibt unberührt und verifiziert weiter. Der akzeptierte Preis: PDF-Reader und DSS melden das Dokument als „nach der Signatur geändert“. Das ist zutreffend, die einzige zulässige Ursache ist dieser Re-Anchor, und die eigene Verifikation weist genau das nach (Kapitel 07).

Multi-Signer. Deklariert ein Vertrag mehrere Signaturfelder, muss für jedes Feld eine verifizierte Ceremony existieren, bevor die erste Signatur angebracht wird: Die Evidenz aller Signaturen muss im PDF stecken, bevor eine PAdES-Signatur es einfriert. Ein bereits signiertes Feld kann nicht erneut signiert werden.

QR-/Deep-Link-Variante (OID4VP Document Retrieval). Für vollständig wallet-getriebenes Signieren publiziert die Ceremony ein signiertes Request-Objekt nach dem EUDI-„wallet-driven-signer“-Profil mit zwei Dokumenten, dem zu signierenden PDF und der gepinnten JSON-LD-Payload, so dass eine Ceremony PAdES und JAdES über denselben Inhalt liefert. Die Wallet betreibt ihre eigene SCA/QTSP-Strecke und postet die signierten Dokumente per `direct_post` an den Callback, der denselben Validierungspfad durchläuft; die Antwort muss die frische Request-Nonce der Ceremony binden (6.4).

6.3 Signaturniveaus und -formate

Das geforderte Niveau ist eine Eigenschaft des Vertrags, nicht des Aufrufers. Jedes Signaturfeld deklariert es im Dokument; fehlt die Angabe, gilt AES. Das Niveau wird bei der Vorbereitung gepinnt und bei der Einreichung gegen das **tatsächlich erreichte** Niveau durchgesetzt: Eine gültige AES auf einem QES-pflichtigen Feld wird abgelehnt, und aufgezeichnet wird das erreichte Niveau (ADR-20). Verglichen wird auf der Skala $SES < AES < QES$; wer eine Anforderung durchsetzen will, fordert AES oder QES.

v1-Leistungsumfang (ADR-24). Die Architektur ist nicht AES-limitiert; das erreichte Niveau bestimmen Credential und Vertrauensdienst. Mit den testgradigen v1-Providern (projekteigene CA, Test-Wallet, Demonstrations-TSA, DSS-Testinstanz) ist das Ergebnis eine wallet-basierte AES. QES ist aus dem v1-Liefergegenstand ausgenommen; Verträge mit gesetzlichem Schriftformerfordernis sind in v1 nicht demonstrierbar. Der mitgelieferte Beispiel-PID-Aussteller führt keine Identitätsprüfung der Person durch; sein Credential ist kein EUDI-PID und darf nicht als solches behandelt werden (ADR-31).

Die beiden Formate:

- **PAdES** ist die Signatur des Signatars über das menschenlesbare PDF, sichtbar im AcroForm-Feld, extern erzeugt.
- **JAdES** (ETSI TS 119 182-1, Baseline-B) ist das Gegenstück über die maschinenlesbare Repräsentation: ein kompaktes JWS über die kanonisierte Form aus Vertrags-DID, Version und vollständigem JSON-LD-Dokument. Bei einer publizierten Wallet-Ceremony ist die JAdES des Signatars Pflicht, weil sie die Nonce-Bindung trägt; auf dem authentifizierten Einreichungspfad ist sie optional und wird, falls vorhanden, validiert.

Unabhängig davon signiert die **Instanz** jeden an Peers verschickten Vertrag mit ihrem HSM-gestützten DID-Schlüssel als JAdES (Kapitel 08). Diese Instanzsignatur ist eine Systemhandlung, keine AES einer Person (ADR-17). Ein elektronisches **Siegel** einer Organisation ist als Richtung entschieden, aber nicht implementiert (ADR-21).

6.4 Validierung: interne Prüfungen plus EU DSS

Eingereichte Signaturen durchlaufen eine mehrstufige Annahmeprüfung (ADR-20); jede Stufe ist ein hartes Abbruchkriterium:

1. **Byte-Pinning.** Das eingereichte PDF muss byte-genau mit dem gepinnten TBS-PDF **beginnen**: Eine PAdES-Signatur ist ein inkrementelles Update, das nur Bytes anhängt. Die JAdES-Payload wird genauso gegen die gepinnte kanonische Payload geprüft.
2. **Inhaltsvergleich.** pdf-core vergleicht den sichtbaren Seiteninhalt des eingereichten Dokuments mit dem vorbereiteten; nichts wird neu gerendert.

3. **Nonce-Bindung.** Bei einer publizierten Wallet-Ceremony muss die JAdES im signaturgeschützten Header die frische Request-Nonce der Ceremony tragen. Wer nur die Ceremony-URL kennt, kann ohne den Schlüssel des Signatars keine annehmbare Antwort fälschen.
4. **Extern über DSS.** Der Report liefert die ETSI-Indication, Qualification, Format/Level, Zertifikats-Subject und Signierzeit, bei Mehrfachsignaturen der jüngsten Signatur zugeordnet. **Ohne konfigurierten DSS nimmt der Einreichungspfad keine Signatur an**; auf den Lesepfad entfällt ohne DSS lediglich der Report.
5. **Level-Gate.** Für **AES** ist das Kriterium bewusst nicht `TOTAL-PASSED`, denn das verlangt zusätzlich eine qualifizierte EU-Trust-List-CA, also eine QES-Eigenschaft. Für AES genügen kryptographische Integrität und eindeutige Zuordnung; ein `INDETERMINATE` mit reiner Trust-Chain-Lücke wird akzeptiert, jeder Krypto- oder Integritätsfehler abgelehnt. Für **QES** gilt `TOTAL-PASSED` **und** die Qualification eines qualifizierten Zertifikats mit QSCD.
6. **Sole-Control-Gate.** Das Signaturzertifikat wird direkt aus der CMS-Struktur des eingereichten PDFs gelesen; sein Subject muss dem Namen der verifizierten Identität der Ceremony entsprechen. Für QES ist der Abgleich zwingend (eIDAS Annex I), für AES standardmäßig aktiv und nur als dokumentierte Betreiberentscheidung abschaltbar. Derselbe Signatar muss über alle eigenen Signaturen eines Vertrags dasselbe Zertifikat verwenden. Subject und Seriennummer werden für den Compliance Viewer aufgezeichnet.

Ein konfigurierter, aber nicht erreichbarer DSS ist ein Fehler, keine übersprungene Prüfung. Eine ungültige JAdES wird abgelehnt, nie stillschweigend mitaufgezeichnet.

Nachträgliche Prüfung. Ein signierter Vertrag lässt sich jederzeit erneut prüfen. Die Prüfung liest die eingebetteten Signing-Summary-Credentials und verifiziert sie zuerst gegen den Schlüssel ihres Ausstellers. Darauf setzen auf: die **Signatarbindung** (der pseudonyme Identifier aus der Evidenz gegen die aufgezeichneten Signaturen), die **SHACL-Drift-Prüfung** (Wiederholung der Validierung gegen exakt die Shapes-Version der Signatur und Vergleich des Befund-Hashes; ein Hub-Versionssprung erzeugt keinen Fehlalarm, ADR-8) und die **DSS-Bewertung**, sofern konfiguriert.

6.5 Zeitstempelung

Mit `SIGNED` entsteht der Archiv-Eintrag; dessen Notarisierungs-Evidenz erhält einen RFC-3161-Zeitstempel. Die Anfrage läuft über ORCE, das Backend verifiziert die Antwort gegen das vertrauenswürdige TSA-Zertifikat. Ein TSA-Wechsel bedeutet deshalb zweierlei: den Flow in ORCE umstellen und den Vertrauensanker austauschen. Dieselbe Maschinerie nutzen die Audit-Checkpoints; eine vorübergehend nicht erreichbare TSA wird nachgeholt (Kapitel 09).

6.6 Sichtbarkeit: Viewer, Verifikation, Widerruf

- **Signierliste:** ausstehende, abgeschlossene und widerrufene Verträge des Signatars; jede Signaturanwendung hinterlässt einen Audit-Eintrag.
- **Secure Contract Viewer:** Integritätsprüfung (Neuaufbau des Basis-PDF, Hash-Abgleich, C2PA- und Widerrufsprüfung) und Einstieg in die Ceremony; die C2PA-Kette ist über einen eigenen Abruf einsehbar.

- **Signature Compliance Viewer:** pro Signatur Signatar, Feld, gefordertes und erreichtes Niveau, Zertifikats-Subject und Seriennummer (Sole-Control-Evidenz), die kryptographischen Bindungen aus dem Signing-Summary-Credential sowie Integritätsbefunde und den DSS-Report.
- **Widerruf:** setzt eine Signatur auf `REVOKED`; der Envelope behält beide Zeitpunkte, der Vorgang ist auditiert. Bei einem grenzüberschreitenden Vertrag geht der Widerruf unmittelbar an die Gegenpartei, die den Zustand übernimmt (Kapitel 08).

6.7 Bekannte Grenzen

- **Extraktion des eingebetteten JSON-LD.** Der Abgleich löst das eingebettete Dokument über die letzte Definition des Anhangsobjekts im Dateiinhalt auf, nicht über das Querverweisverzeichnis. Ein gezielt präpariertes Dokument kann diese Auflösung von der eines PDF-Readers abweichen lassen.
- **Mehr als zwei Parteien.** Der Nachweis der Gegenseiten-Signatur wird je Vertrag vorgehalten, nicht je Signaturfeld. Verträge mit drei oder mehr Parteien werden deshalb nicht ausgerollt (Kapitel 04).
- **„Nach der Signatur geändert“.** Prüfprogramme melden ein DCS-signiertes Dokument als nach der Signatur verändert. Die Signatur bleibt gültig; das Verhalten ist die bewusste Folge des Provenance-Re-Anchors (ADR-26) und wird von der eigenen Verifikation nachgewiesen (Kapitel 07).

07 Dokumente und Provenance

Dieses Kapitel beschreibt, wie aus dem maschinenlesbaren Vertrag (JSON-LD) das menschenlesbare Vertragsdokument (PDF/A-3) entsteht, warum das Rendering deterministisch ist, wie jede Instanz unabhängig prüft, dass beide Repräsentationen übereinstimmen, wie die Herkunft über C2PA-Manifeste nachvollziehbar bleibt und wie Artefakte gespeichert und gelöscht werden. Das PDF ist kein Ausdruck des Vertrags, sondern eine zweite, jederzeit gegen die maschinenlesbare Form verifizierbare Repräsentation derselben Vereinbarung.

7.1 Beteiligte Komponenten

Komponente	Rolle
pdf-core	Eigenständiger, zustandsloser Dienst; alleiniger Eigentümer aller Byte-Arbeit am PDF: deterministisches Kompilieren, inkrementelle Amendments, Provenance-Re-Anchor, Extraktion des eingebetteten JSON-LD, C2PA-Einbettung, Re-Render- und Inhaltsvergleich. Das Backend parst nie selbst PDF-Bytes
Backend / PDF-Generierung	Orchestriert: hört auf Lifecycle-Ereignisse, lässt pdf-core rendern bzw. amendieren, stellt Lifecycle-Credentials aus, signiert die C2PA-Strukturen, legt das Ergebnis verschlüsselt ab und führt den PDF-Zustand je Vertrag/Vorlage (CID, Renderer-Version, C2PA-Zustand, Payload-Hash)
Artefaktspeicher	Verschlüsselnde Schicht über IPFS: jedes Artefakt wird vor dem Schreiben verschlüsselt, jeder Lesevorgang entschlüsselt authentifiziert
C2PA-Manifest-Dienst	Öffentlicher, unauthentifizierter Abruf des C2PA-Manifest-Stores eines Vertrags
HSM (über das Backend)	Signiert die COSE-Strukturen der C2PA-Manifeste und entpackt die Inhaltsschlüssel. pdf-core hält kein Schlüsselmaterial: Es baut die zu signierende Struktur, das Backend signiert
Statuslist-Dienst	Liefert bei der Verifikation den Live-Widerrufsstatus der in den Manifesten eingebetteten Lifecycle-Credentials

7.2 Deterministisches Rendering

pdf-core garantiert: **Dieselbe JSON-LD-Payload erzeugt immer byte-identischen Seiteninhalt.**

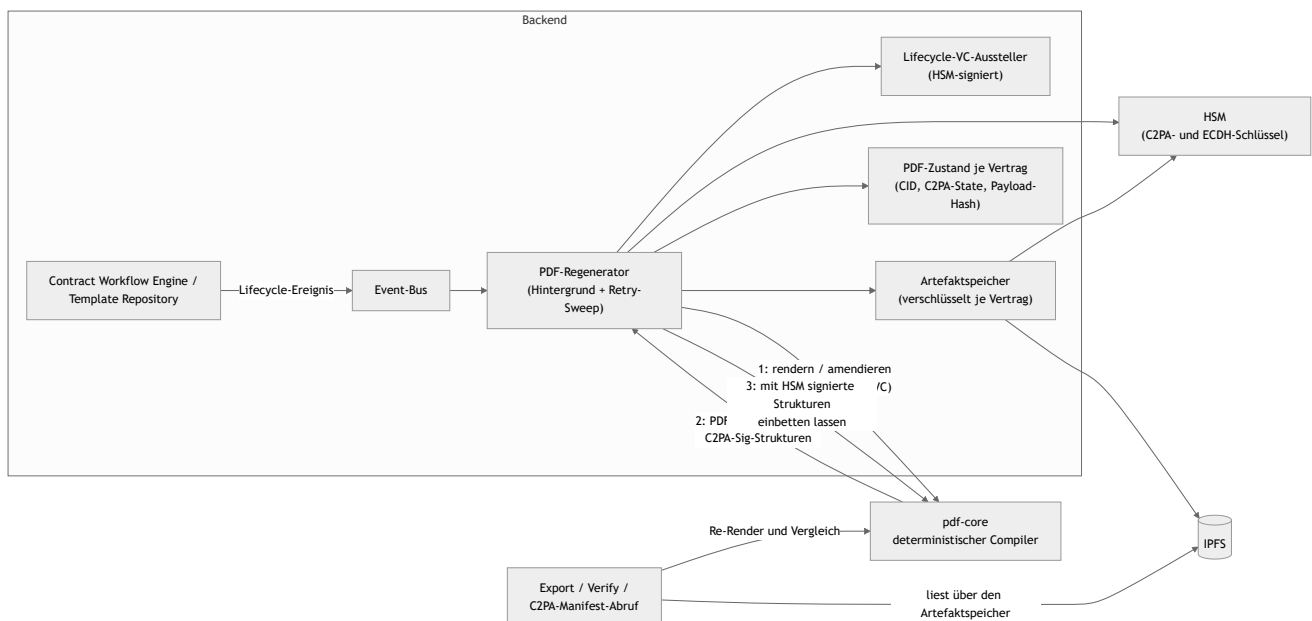
- Der Render-Zeitpunkt ist eine feste Epoche, nicht die Wanduhr; die vertrauenswürdige Vertragszeit ist der Zeitstempel der PAdES-Signatur (Kapitel 06).
- Die eingereichte Payload wird **verbatim** als Anhang eingebettet, exakt die übermittelten Bytes. Ihr SHA-256 ist die Content-Adresse des Dokuments, für jeden Prüfer aus dem Anhang nachrechenbar.
- Schrift ist eingebettet (PDF/A-Konformität); Layout und Struktur sind vollständig aus der Payload abgeleitet. Semantische Zusatzdaten, die nicht gerendert werden (etwa ODRL-Policies), bleiben unverändert im eingebetteten JSON-LD erhalten.

Weil der Seiteninhalt eine reine Funktion der Payload ist, lässt sich die Übereinstimmung von menschen- und maschinenlesbarer Form jederzeit beweisen: Payload extrahieren, neu kompilieren, Seiteninhalte vergleichen.

Mehrfache Signaturen sind konstruktionsbedingt **sequenziell**: PDF/A-3 verlangt, dass jede Signatur alle vorherigen Bytes abdeckt. Änderungen nach einer Signatur erfolgen als inkrementelle Revisionen über den signierten Bytes.

7.3 Der Rendering- und Provenance-Fluss

PDFs entstehen nie auf Zuruf im Request-Pfad, sondern ereignisgetrieben im Hintergrund. Jedes Lifecycle-Ereignis läuft über den Event-Bus in den Regenerator: Der stellt ein Lifecycle-Verifiable-Credential aus, lässt pdf-core rendern bzw. amendieren (die C2PA-Kette wächst, sie beginnt nie neu), signiert die zurückgelieferten C2PA-Strukturen mit dem HSM und lässt sie einbetten, legt das Ergebnis verschlüsselt ab und aktualisiert den PDF-Zustand.



Der Export bedient sich immer aus diesem Bestand: Er liefert das aktuelle PDF und wartet, wenn nach der letzten Änderung noch eine Regeneration läuft. Er gibt nie einen veralteten Stand und rendert nie selbst. Ein bereits PAdES-signiertes PDF ist **eingefroren**: Es wird unverändert ausgeliefert, auch wenn der Lifecycle-Zustand weiterläuft. Die einzige Revision nach der Signatur ist der Provenance-Re-Anchor der Signatur-Finalisierung (ADR-26, Kapitel 06); danach mutiert nichts mehr.

7.4 C2PA-Manifeste

Jede PDF-Version trägt einen C2PA-Manifest-Store (JUMBF), der den gesamten sichtbaren Seiteninhalt abdeckt. Die Manifeste **stapeln** sich über den Lebenszyklus: Jeder Zustandswechsel hängt eine Lifecycle-Assertion an, zusammen mit einem Verifiable Credential der Instanz, das den Wechsel, den Asset-Hash und den Zeitpunkt bezeugt und über eine Statusliste widerrufbar ist. Die COSE-Signatur jedes Manifests erzeugt das Backend mit dem HSM-gehaltenen C2PA-Schlüssel; die Zertifikatskette ist konfiguriert und wird pdf-core als öffentliches Material bereitgestellt.

Ein signierter Vertrag trägt ein Manifest mehr als ein unsignierter: das Lifecycle-Manifest, auf das sich die Signatur festlegt, und das Re-Anchor-Manifest, das sich auf die Signatur festlegt (ADR-26).

Ausgeliefert wird die Provenance doppelt (ADR-4): eingebettet als JUMBF im PDF und über den öffentlichen C2PA-Endpunkt, denn ein externer Prüfer muss die Herkunftskette ohne Konto auflösen können. Damit hängt die Sichtbarkeit einer Kette zugleich an der Verfügbarkeit ihres Inhaltsschlüssels (7.6).

7.5 Unabhängige Validierung: menschen- vs. maschinenlesbar

Die Konsistenz beider Repräsentationen wird an drei Stellen erzwungen:

1. **Auf Abruf** über die Verify-Endpunkte: eingebettetes JSON-LD extrahieren, neu rendern, Seiteninhalt gegen den gespeicherten Stand vergleichen; zusätzlich C2PA-Signatur, Lifecycle-Credential und dessen Live-Widerrufsstatus prüfen.
2. **Beim Empfang eines Peer-PDFs** (Kapitel 08): Der Seiteninhalt muss exakt der Re-Render der eigenen eingebetteten Payload sein. Der Vergleich betrachtet nur die Seiteninhalte, so dass legitime C2PA-, Signatur- und Amendment-Schichten des Absenders nicht anschlagen.
3. **Bei der Signaturannahme** (Kapitel 06): eingereichtes gegen vorbereitetes Dokument.

Bei einem PDF mit angehängten Revisionen spielt die Verifikation jede Revision deterministisch nach: eine Revision mit neuem Lifecycle-Credential als Amendment, eine Revision mit unveränderter Payload als Provenance-Re-Anchor. Ein signiertes PDF gilt nur dann als gut, wenn das einzige nach der Signatur Angehängte byte-genau die Provenance ist, die diese Instanz selbst produziert hat. Die „nach der Signatur geändert“-Meldung der PDF-Reader wird nachgewiesen, nicht ignoriert.

Das Verifikationsergebnis ist ehrlich geschnitten. Trägt das PDF keine PAdES-Signatur oder prüft dieser Pfad sie nicht kryptographisch, meldet das Ergebnis für die PDF-Signatur „nicht verfügbar“ statt einer scheinbar bestandenen Prüfung. Zusätzlich benennt es seine Fehlerklasse direkt (ADR-30):

Klasse	Bedeutung
content_hash_mismatch	Manifest vorhanden, aber der Inhalt weicht vom eingebetteten JSON-LD ab
artifact_not_authentic	Die gespeicherten Bytes haben die authentifizierte Entschlüsselung nicht bestanden; sie sind nicht die, die diese Instanz geschrieben hat
verification_failed	Eine andere Prüfung ist fehlgeschlagen
(leer)	Die Prüfung ist bestanden

7.6 Speicherung: Verschlüsselung, Manipulationsnachweis, Löschung

Jedes Artefakt wird **vor** dem Schreiben verschlüsselt (ADR-28). Der Inhaltsschlüssel ist pro **Löschbereich** zufällig gezogen: je Vertrag, je Vorlage, und instanzweit für Checkpoints und instanzbezogene Reports. Der Bereichsbezeichner geht als authentifizierte Angabe in das Chifftrat ein, so dass ein Artefakt nicht unbemerkt einem anderen Bereich untergeschoben werden kann.

Der Inhaltsschlüssel existiert im Ruhezustand **nur gewickelt** gegen den im DID-Dokument publizierten Schlüsselvereinbarungs-Schlüssel; Auspacken erfordert das HSM genau dieser Instanz. Bei jeder Vertragssendung reist eine für den Peer gewickelte Kopie mit; der Empfänger übernimmt sie einmal und wickelt sie gegen den eigenen Schlüssel neu. Beide Instanzen halten danach denselben Inhaltsschlüssel, jede Kopie nur vom eigenen HSM zu öffnen.

Drei Folgen:

- **Manipulation am Speicher ist ein Prüfergebnis, kein Serverfehler (ADR-30).** Scheitert die authentifizierte Entschlüsselung, sind die gespeicherten Bytes nicht die geschriebenen; genau das meldet die Verifikation. Klartext wird nie aus unauthentifiziertem Chifftrat abgeleitet.
- **CIDs zweier Instanzen unterscheiden sich** für denselben Vertrag, weil jede Seite unter eigenem Zufallswert verschlüsselt. Das ist folgenlos: Über die Peer-Schnittstelle wandert nie eine CID.
- **Löschung ist Schlüsselvernichtung.** Erasure zerstört die gewickelten Kopien und behält die Zeile als Zerstörungsnachweis (Kapitel 04 und 09). Export, Verifikation und Bundle-Abruf liefern danach eine definierte „Inhalt gelöscht“-Antwort; Listen- und Metadatenansichten arbeiten weiter.

Bekannte Grenze. Die Schlüsselvernichtung entfernt die Chifftrate nicht aus dem IPFS-Knoten. Solange eine Datenbanksicherung von vor der Vernichtung existiert und der Schlüsselvereinbarungs-Schlüssel im HSM weiterlebt, ließe sich daraus wieder ein lesbarer Schlüssel gewinnen. Die Löschzusage reicht so weit wie das Aufbewahrungsfenster der Sicherungen; das Nachziehen der Löschmarkierungen nach einer Wiederherstellung beschreibt der Backup-Leitfaden.

7.7 Schnittstellen und Artefakte

Backend (authentifiziert, rollenbasiert; vgl. Anhang A):

Endpunkt	Zweck
GET /pdf/export/contract/{did} , GET /pdf/export/template/{did}	Aktuelles PDF (PDF/A-3, eingebettetes JSON-LD, C2PA-Kette)
GET /pdf/verify/contract/{did} , GET /pdf/verify/template/{did}	Text/Daten- und Provenance-Verifikation inklusive Fehlerklasse
GET /contract/export/{did}	ZIP-Bundle: JSON-LD, signiertes PDF, C2PA-Store, Credentials, Signaturzustände, Manifest mit Hash je Eintrag, Hierarchie-Verwandte
GET /template/export/{did}	ZIP-Bundle einer Vorlage
GET /c2pa/manifest/{contract_did}	Öffentlicher C2PA-Manifest-Store; optional die Kettenaufzählung

pdf-core (intern, ausschließlich vom Backend angesprochen), nach Aufgabengruppen:

Gruppe	Endpunkte	Zweck
Rendern	POST /render , /render/amendment , /render/reanchor	Kompilieren, inkrementell fortschreiben, Provenance-Re-Anchor (ADR-26)
Prüfen	POST /verify , /verify/content , /verify/content-match	Re-Render-Verifikation; Empfangsprüfung für Peer-PDFs; Vergleich zweier PDFs bei der Signaturannahme
Extrahieren und Binden	POST /payload/extract , /manifest/extract , /manifest/chain , /claim	Eingebettetes JSON-LD, C2PA-Store und Kettenaufzählung auslesen; externe Payload gegen ein PDF binden
Einbetten	POST /c2pa/embed , /evidence/embed , /evidence/extract	Vom Backend signierte COSE-Strukturen einsetzen; Signatur-Evidenz ein- und auslesen
Meta	GET /version , GET /ontology/...	Renderer-Version (Teil des persistierten PDF-Zustands) und Schema-Artefakte des Renderers

Die Verbindungsdaten von pdf-core sind Deployment-Konfiguration (Deployment-Leitfaden).

7.8 Betriebsverhalten

- **Export wartet statt zu lügen:** Läuft noch eine Regeneration, pollt der Export und bricht nach seinem Zeitfenster mit einem klaren Fehler ab; die blockierende Bedingung steht im Log.
- **Verify setzt einen vorhandenen Bestand voraus:** Ohne vorherigen Export schlägt die Verifikation mit entsprechender Meldung fehl. Ist der Lifecycle-Zustand fortgeschritten, regeneriert sie transparent.
- **Verlorene Regenerationen holt ein Sweep nach:** Ein periodischer Durchlauf findet Entitäten, deren gespeichertes PDF nicht dem aktuellen Dokument entspricht, arbeitet sie in begrenzten Stapeln ab und wiederholt jede nur begrenzt oft mit wachsendem Abstand. Ein dauerhaft unrenderbarer Vertrag blockiert die heilbaren nicht.
- **IPFS ist eventual consistent:** Frisch geschriebene CIDs sind nicht immer sofort lesbar; der Client wiederholt mit Backoff.
- **Bundle-Exporte verweigern statt unvollständig zu liefern:** Fehlt eine referenzierte Komponente, antwortet der Export mit einer Befundliste statt eines lückenhaften Archivs.
- **Ein Amendment ohne inhaltliche Änderung** wird abgelehnt; die bewusste Ausnahme ist der Provenance-Re-Anchor mit seinem eigenen Endpunkt.
- **Ein fehlerhaftes Peer-PDF ist ein Ablehnungsgrund, kein Absturz:** Die Verarbeitung eingehender Dokumente ist in Größe und Aufwand begrenzt.

08 Föderation

Dieses Kapitel beschreibt, wie zwei unabhängige DCS-Instanzen Verträge austauschen und verhandeln, welches Trust-Modell jede Interaktion absichert und wie Vorlagen über den XFSC Federated Catalogue auffindbar werden. Das Grundprinzip: Über die Instanzgrenze wandern **Artefakte**, **kein Zustand**. Jede Instanz führt ihren eigenen Workflow, ihre eigene RBAC und ihre eigene Kopie des Vertrags (ADR-13).

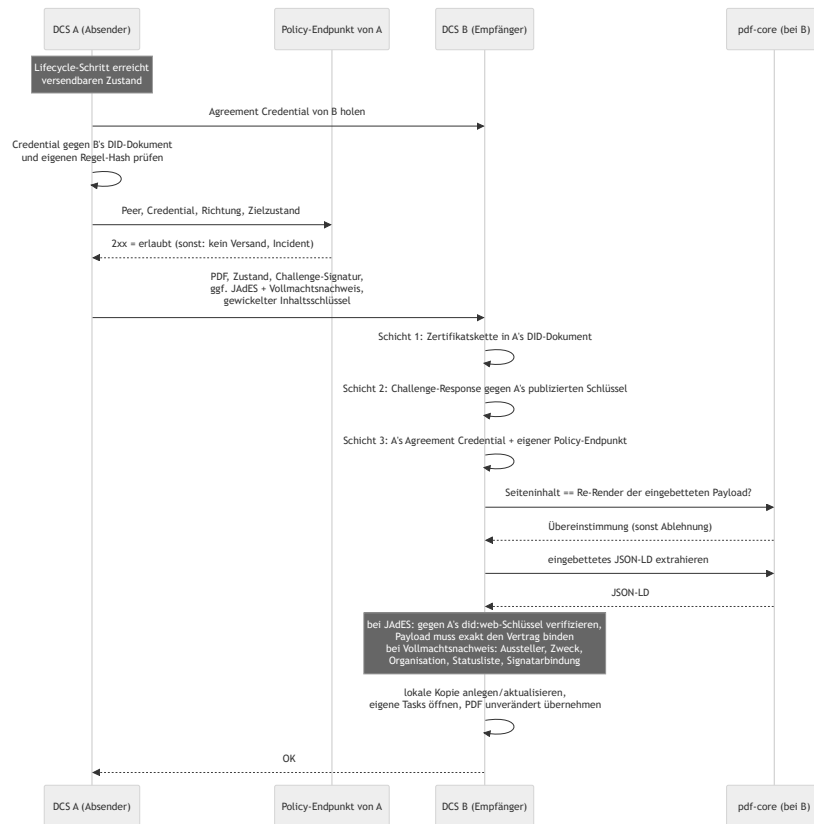
8.1 Beteiligte Komponenten

Komponente	Rolle
DCS-to-DCS-Synchronizer	Ausgehende Seite: hört auf Lifecycle-Ereignisse und verschickt das Vertrags-PDF; Fehlversuche landen in einer Retry-Tabelle
Peer-Endpunkte	Eingehende Seite: authentifizieren den Absender, prüfen das PDF (und bei signierten Sendungen JAdES und Vollmachtsnachweis), übernehmen es als lokale Kopie; nehmen Löschanforderungen entgegen
Trust Gate	Vertrauensschicht vor jeder Interaktion, in beide Richtungen: Agreement Credential des Peers und lokaler Policy-Endpunkt (fail-closed)
Counterparty-PoA-Gate	Prüft die Handlungsvollmacht hinter jeder Signatur, die eine Gegenpartei mitschickt (ADR-31)
did:web-Identität + HSM	Instanzidentität mit Zertifikatskette; private Schlüssel im HSM (Kapitel 05)
pdf-core	Prüft beim Empfang, dass der Seiteninhalt der Re-Render der eingebetteten Payload ist, und extrahiert das JSON-LD (Kapitel 07)
Audit-Trail	Jede Trust-Gate-Ablehnung wird als Incident erfasst (Kapitel 09)
Federated Catalogue	Instanzübergreifende Vorlagen-Discovery

8.2 Instanz-zu-Instanz-Austausch: das PDF ist das Wire-Format

Ausgetauscht werden das **Vertrags-PDF pro sichtbarem Lebenszyklusschritt**, die **JAdES-Signatur nach dem Signieren** und der **Nachweis der Handlungsvollmacht** hinter dieser Signatur. Das PDF ist selbsttragend: Es enthält das JSON-LD, die C2PA-Kette und etwaige Signaturen. Ein bloßes PDF ist ein Vorschlag, ein PDF mit JAdES die Signatur der Gegenseite. Zusätzlich reist der für den Peer gewickelte Inhaltsschlüssel mit (ADR-28, Kapitel 07).

Verschickt wird nur, was die Gegenseite sehen muss: die Zustände `OFFERED`, `NEGOTIATION`, `SIGNED` und `REVOKED`. Interne Zustände (Entwurf, Review, Freigabe, Aktivierung, Terminierung) bleiben lokal. Die Verhandlung verläuft in drei Phasen: verhandeln (Gegenvorschläge als neue PDF-Versionen), einigen (beide Seiten bestätigen dieselbe Version) und erst danach signieren; ein einseitig signierter, noch nicht geeinigter Vertrag wäre ein disputables Artefakt.



Beim **ersten Empfang** entsteht die lokale Kopie im Zustand **OFFERED**, mit dem Absender als Origin; die eigenen Review-, Freigabe- und Verhandlungs-Tasks werden lokal geöffnet. Ein **erneuter Empfang** aktualisiert den Inhalt und erhöht die lokale Version, ohne den eigenen Workflow-Fortschritt zu überschreiben. Der vom Absender deklarierte Zustand ist rein informativ, mit genau einer Ausnahme: Meldet die authentifizierte Gegenpartei **REVOKED**, übernimmt der Empfänger diesen Zustand, denn der Widerruf entwertet die Vereinbarung.

Das empfangene PDF wird **exakt so, wie es kam**, als eigene Kopie gespeichert. Ein Neu-Rendern würde die C2PA-Kette der Gegenseite zerstören; spätere eigene Änderungen amendieren diese Basis, so dass die Kette über beide Instanzen hinweg wächst.

Transport folgt der Identität. Der Peer wird aus seinem **did:web**-Identifizier aufgelöst, einschließlich Pfadsegmenten, so dass mehrere Instanzen einen Host teilen können. Die Auflösung ist HTTPS-only; die einzigen Ausnahmen sind Loopback-Hosts und Hosts, die ein Deployment ausdrücklich als unsicher benennt (clusterinterne Identitäten). Ausgehende Abrufe folgen keinen Weiterleitungen, wählen keine Adressen, an denen eine publizierte Identität nie liegt, und sind mit Timeouts begrenzt. Authentifiziert wird nicht auf Transportebene, sondern per Challenge-Response-Signatur im Request selbst.

8.3 Trust-Modell zwischen Instanzen

Vertrauen entsteht ausschließlich aus verifizierbaren Schichten:

Schicht	Frage	Mechanismus	Anker
1: Identität	Wer bist du?	did:web mit Zertifikatskette, optional gegen den EU-Trust-Pool geprüft	EU-Vertrauenslisten / QTSP
2: Besitznachweis	Sprichst gerade du?	Challenge-Response pro Request: Zufallswert, mit dem HSM-Schlüssel des Absenders signiert, verifiziert gegen dessen publizierten Schlüssel	Schicht 1
3a: Regelakzeptanz	Spielst du nach den Regeln?	Selbstsigniertes Agreement Credential unter <code>/.well-known/dcs-agreement-credential.json</code> , dessen <code>termsOfUse</code> - Hash das Föderationsregelwerk benennt	In die Binärdatei einkompiliertes Regelwerk; der Hash muss dem eigenen entsprechen
3b: Policy	Darf diese Interaktion stattfinden?	Eigener lokaler Policy-Endpunkt (<code>DCS_TRUST_PDP_URL</code>): 2xx erlaubt, alles andere verweigert	Was auch immer der Endpunkt konsultiert (Allowlist, Trust-Listen, Clearing House, ...)
3c: Vollmacht	Durfte der Unterzeichner für diese Partei handeln?	Mitgeschickter Vollmachtsnachweis, geprüft gegen einen für <code>peer</code> zugelassenen Aussteller mit Organisationsberechtigung, inklusive Statusliste (ADR-31)	Ausstellervertrauen der empfangenden Instanz
4: Rechtswirkung	Bindet der Vertrag?	AES/QES auf dem Dokument (PAdES/JAdES, Kapitel 06)	eIDAS

Die Schichten 3a/3b bilden das **Trust Gate** (ADR-19), konsultiert vor jedem Versand und bei jedem Empfang:

- **Fail-closed.** Ein nicht konfigurierter, nicht erreichbarer oder nicht antwortender Policy-Endpunkt verweigert wie ein explizites Deny. Ohne konfigurierten Policy-Endpunkt ist Föderation wirksam abgeschaltet.
- **Regelakzeptanz ist an die Softwareversion gebunden.** Das Regelwerk ist einkompiliert und unter `/.well-known/dcs-federation-rules.md` abrufbar; Regeländerungen sind Software-Releases.
- **Das Credential muss vom Peer selbst stammen.** Aussteller und Bezugsquelle müssen auf dasselbe Auflösungsziel zeigen, und der Proof muss einen Assertion-Schlüssel des Peer-DID-Dokuments referenzieren.
- **Kein „Phone home“.** Jede Instanz befragt ausschließlich ihren eigenen Policy-Endpunkt. Die Standard-Auslieferung enthält einen minimalen Flow, den Betreiber erweitern.
- **Jede Ablehnung ist auditierbar.** Auf dem Versandpfad wird eine Agreement-Credential-Ablehnung erneut versucht; eine Policy-Endpunkt-Ablehnung ist terminal: kein Retry, genau ein Incident pro Versuch, dedupliziert pro Vertrag, Peer und Richtung.

Provenance und Vollmacht des Gegenseiten-Artefakts. Die JAdES eines signierten Versands wird vor der Annahme vollständig verifiziert: gegen die eigene Zertifikatskette, mit dem publizierten `did:web` - Assertion-Schlüssel des Absenders als Leaf, und die signierte Payload muss exakt den im PDF eingebetteten Vertrag binden. Die Challenge-Response authentifiziert nur die Sitzung; die JAdES bindet den Vertrags**inhalt** an den Schlüssel des Absenders und bleibt als unabhängig nachprüfbarer Nachweis abrufbar.

Der mitgeschickte Vollmachtsnachweis (ADR-31) wird geprüft, bevor etwas persistiert wird: für **peer** zugelassener Aussteller mit Berechtigung für diese Organisation, gültige Signatur und Frist, nicht widerrufen, autorisiert genau die Partei, die signiert hat, und gehalten von genau dem ausgewiesenen Unterzeichner. Was die Prüfung feststellt, ist eine Attestierung von Vollmacht; wer der Mensch ist, hat die Instanz festgestellt, auf der die Ceremony lief. Das Fehlverhalten ist bewusst asymmetrisch: Ein **vorhandener, nicht verifizierbarer** Nachweis verweigert die Sendung und erzeugt einen Incident; ein **fehlender** nicht, denn ein Peer ohne aufbewahrte Nachweise muss weiter fördern können, und eine Partei ohne Vollmacht weist der Compliance Viewer aus. Läuft ein Vollmachts-Credential nach der Signatur ab oder wird es widerrufen, scheitern spätere Sendungen desselben Vertrags; die Lebensdauer einer Vollmacht muss den Austausch überdauern.

8.4 Löschung über die Instanzgrenze

Der Löschpfad ist ein eigener Peer-Aufruf mit derselben did:web-Challenge-Response wie der PDF-Austausch:

1. Die anfragende Instanz vernichtet ihre eigenen gewickelten Inhaltsschlüssel und schreibt ein Zerstörungseignis in den Audit-Trail. Scheitert dieser lokale Schritt, ist das ein harter Fehler.
2. Sie fordert jede Gegenpartei-Instanz auf, dasselbe zu tun. Eine nicht erreichbare Gegenseite blockiert nicht: Die Anforderung bleibt offen und wird periodisch erneut zugestellt.
3. Beide Seiten emittieren dasselbe Zerstörungseignis, mit Akteur, Vertrag, Bereich und Grund, nie mit Inhalt.

Ein vernichteter Bereich ist endgültig: Weder ein lokaler Schreibvorgang noch ein vom Peer mitgeschickter Schlüssel erzeugt einen neuen. Der Fortschritt der Kette ist über eine Statusabfrage sichtbar (Kapitel 09); die Grenzen der Zusage beschreibt Kapitel 07.

8.5 Federated-Catalogue-Integration

Der Federated Catalogue dient der Vorlagen-Discovery, nicht der Geschäftsdatenhaltung: Er verifiziert und speichert Self-Descriptions und macht sie abfragbar; mehrere FC-Instanzen können sich synchronisieren. Das Datenmodell verknüpft die Vorlage (Resource) mit dem Participant und einem ServiceOffering, das die Endpunkt-URL der betreibenden DCS-Instanz trägt. Von einer gefundenen Vorlage lässt sich so zum zuständigen DCS navigieren; die übliche Übernahme ist die Registrierung als eigene Vorlage in das eigene Repository mit eigenem Freigabeprozess (Kapitel 03).

Zur Authentifizierung verwendet die Instanz einen Keycloak-Service-Account, der sämtliche funktionalen Katalogrollen einschließlich der administrativen hält (ADR-18). Daraus folgt eine harte Betriebsgrenze: Ein Katalog darf nicht von mehreren DCS-Deployments geteilt werden, die sich nicht vollständig vertrauen. Das Deployment erzwingt das: Eine Katalog-Integration gegen einen entfernten, nicht mitdeployten Katalog schlägt fehl, solange der Betreiber die gemeinsame administrative Vertrauensgrenze nicht ausdrücklich bestätigt.

Betriebsdetaill der Katalog-Kopplung sammelt der FC-Integrationsleitfaden; das Deployment beschreibt der Deployment-Leitfaden.

8.6 Betriebsverhalten

- **Versand ist ereignisgetrieben und idempotent nachholbar.** Ein fehlgeschlagener Versand hinterlässt einen Retry-Eintrag, den ein Scheduler periodisch erneut versucht; ein Erfolg räumt ihn auf. Diese Zustellung ist datenbankgestützt und übersteht verlorene Event-Bus-Nachrichten.
- **Ein Angebot vor fertigem PDF wird nie stillschweigend verworfen:** Der Versand wird explizit in die Retry-Queue gestellt, bis das PDF vorliegt.
- **Terminale Policy-Denials verlassen die Retry-Queue** atomar mit dem Incident; sie würden sonst endlos erneut anlaufen.
- **Was der Betreiber sieht:** Trust-Gate-Incidents im Audit-Trail mit Peer-DID, Richtung und Begründung; Versand- und Empfangsfehler in den Logs; bei einem Inhalts-Mismatch eine Diagnose mit der divergierenden Stelle; offene Löschanforderungen in der Erasure-Statusabfrage.

09 Audit und Compliance

Das DCS behandelt Nachvollziehbarkeit als kryptographische Eigenschaft: Jedes fachlich relevante Ereignis wird in einem manipulationsnachweisbaren Audit-Trail verankert, dessen Integrität über Hash-Verkettung, Merkle-Checkpoints und vertrauenswürdige Zeitstempel unabhängig prüfbar ist. Dieses Kapitel beschreibt den Audit-Trail, Audits und Reports, das kontinuierliche Compliance-Monitoring und das Policy-Enforcement über ODRL und OPA.

9.1 Beteiligte Komponenten

Komponente	Rolle
Fachkomponenten	Schreiben ihre Ereignisse transaktional in eine Outbox-Tabelle, im selben Datenbank-Commit wie die fachliche Zustandsänderung
Outbox-Prozessor	Publiziert Ereignisse auf dem Event-Bus und verankert audit-sichtbare Ereignisse als Audit-Einträge
Artefaktspeicher über IPFS	Hält Audit-Einträge und Checkpoints content-adressiert; die CIDs sind die Autorität des Trails
PostgreSQL	Index über den Trail: Kettenköpfe, Checkpoints, ausstehende Zeitstempel, Dead-Letter-Status, Risiko-Register, Workflow-Gate-Läufe
TSA (RFC 3161, über einen ORCE-Flow)	Zeitstempel über jede Checkpoint-Root
ORCE, externer Notar	Publiziert die Checkpoints sequenziell an eine externe Senke außerhalb der Reichweite des Betreibers
ORCE, Audit-Executor	Bewertet einen Audit-Abruf: Das DCS sammelt die Evidenz, der Executor bildet die Befunde
ORCE, Workflow-Gate-Executor	Entscheidet vor jedem gate-pflichtigen Übergang über den Vertragsschnappschuss (Kapitel 04)
OPA (eingebettet)	Wertet ODRL-Constraints als Rego-Policies aus
<code>pacmonitor</code> (CronJob)	Führt den Compliance-Sweep planmäßig aus (ADR-32)
Process Audit & Compliance (API)	Audits, Reports, Monitoring, Incident-Reports, Checkpoint-Head und Inklusionsbeweise, Workflow-Gate-Läufe

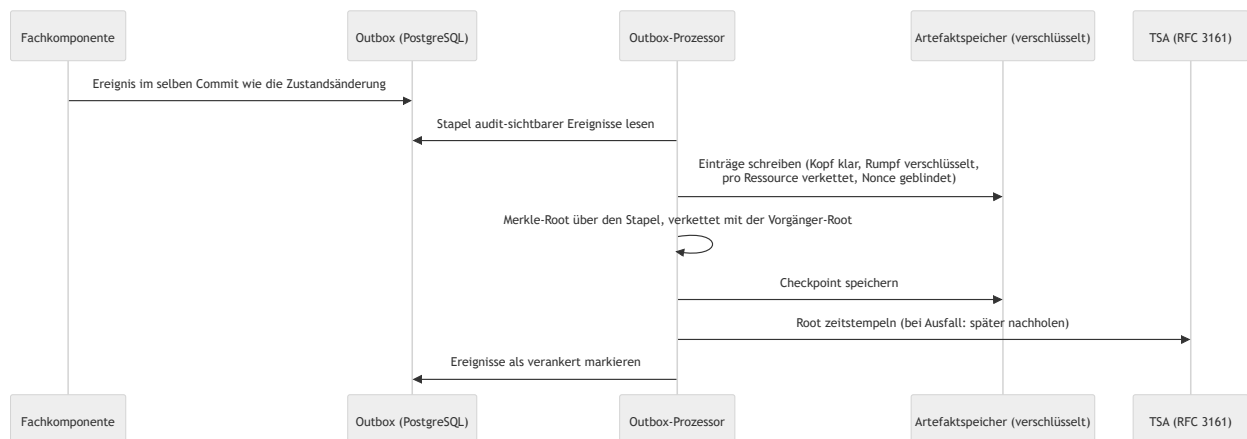
9.2 Der Audit-Trail

Jede fachliche Operation schreibt ihr Ereignis in dieselbe Transaktion wie die Zustandsänderung (Outbox-Muster); ein Ereignis kann nicht verloren gehen, ohne dass auch die Zustandsänderung fehlt. Der Outbox-Prozessor arbeitet in drei entkoppelten Schleifen: **publizieren** (Ereignisse auf den Event-Bus, hoher Takt, wartet nie auf TSA- oder Speicher-Roundtrips), **verankern** (audit-sichtbare Ereignisse stapelweise als Audit-Einträge plus Merkle-Checkpoint; reine Lookup-Ereignisse werden nicht verankert) und **nachzeitstempeln** (Checkpoints, die während einer TSA-Störung ohne Zeitstempel verankert wurden).

Hash-Verkettung pro Ressource. Jeder Audit-Eintrag verweist per CID auf seinen Vorgänger derselben Ressource. Ein nachträglich veränderter Eintrag hätte eine andere CID, und alle Nachfolger zeigten ins Leere. Ketten verschiedener Ressourcen sind unabhängig und werden nebenläufig geschrieben; ein Audit-Lesevorgang läuft die Kette vom Kopf rückwärts.

Öffentlicher Kopf, privater Rumpf (ADR-28). Der Kopf eines Eintrags (Komponente, Ereignistyp, DID, Zeitstempel, Vorgänger-CID, Blinding-Nonce) liegt im Klartext; der Rumpf mit der Ereignis-Payload liegt unter dem Inhaltsschlüssel des jeweiligen Vertrags oder der Vorlage. Ereignisse ohne Ressourcenbezug, einschließlich der Löschnachweise selbst, liegen unter dem instanzweiten Schlüssel, den keine Löschanforderung zerstört. Leaf-Hashes und Checkpoints werden über das gespeicherte Objekt gebildet, wie es ist: Eine Schlüsselvernichtung macht die Rümpfe unlesbar, während jeder Inklusionsbeweis weiter verifiziert. Manipulationsnachweis und Löscharbeit koexistieren.

Merkle-Checkpoints und Zeitstempel (ADR-16). Pro Verankerungs-Stapel wird eine Merkle-Root berechnet (mit Domänentrennung zwischen Blatt- und Knoten-Hashing), verkettet mit der Root des vorherigen Checkpoints: Eine einzige veröffentlichte Root committet transitiv den gesamten Log davor und beweist dessen Append-only-Eigenschaft. Die Root wird einmal pro Stapel zeitgestempelt; bei TSA-Ausfall wird der Checkpoint trotzdem verankert und der Zeitstempel nachgereicht. Jeder Eintrag trägt eine zufällige **Blinding-Nonce** in seinem Blatt-Hash, denn Audit-Einträge sind hochgradig erratbar; mit Nonce sind Inklusionsbeweise gefahrlos publizierbar.



Was publizierbar ist:

Publizierbar	Nie publiziert
Sequenznummer, Root, Vorgänger-Root, Blattanzahl, Erstellzeit, RFC-3161-Token	Blatt-CIDs (Abruffähigkeiten in den eigenen Speicher)
Inklusionsbeweise (ausschließlich Hashes)	Die Einträge selbst (enthalten DIDs, Identitäten, Workflow-Payloads)

Der Beweis-Abruf liefert Blatt-Hash, Blatt-Index, Geschwister-Hashes und den zugehörigen Head, nie den Eintrag, und wird vor der Herausgabe intern gegen die gespeicherte Root verifiziert.

Externe Verankerung. Ein Trail, den nur der Betreiber hält, beweist nichts gegen den Betreiber. Ein ORCE-Flow liest deshalb periodisch die Checkpoints über die Compliance-API (als Maschinen-Identität mit der Rolle **Sys. Auditor**, die nur die Checkpoint-Sicht erreicht) und publiziert die öffentliche Projektion jedes Checkpoints in strikter Sequenz an eine konfigurierte, authentifizierte HTTPS-Senke: lückenlos, mit

Idempotenzschlüssel und persistiertem Bestätigungsstand, so dass eine verlorene Antwort oder ein Neustart keinen Checkpoint doppelt anhängt. Meldet die Senke eine Sequenzlücke oder eine abweichende Vorgänger-Root, stoppt die Publikation sichtbar, statt eine gebrochene externe Kette fortzuschreiben. Ein Dritter verifiziert einen Eintrag mit den Eintrags-Bytes samt Nonce, dem Inklusionsbeweis und einem Checkpoint, den er von der externen Senke bezieht, nicht vom Betreiber. Ohne konfigurierte Senke findet keine externe Verankerung statt.

9.3 Audits und Reports

Audit-Abruf. Auditoren fordern die Audit-Trails eines Scopes an (Templates, Verträge, Archiv, Signaturen; optional auf eine DID gefiltert). Jede Anfrage erfordert eine Begründung, und der Abruf selbst wird als Audit-Ereignis in den Trail geschrieben. Der Abruf ist zweistufig: Das DCS sammelt die Evidenz (Hash-Ketten plus frisch berechnete Inhalts- und Policy-Prüfungen) und übergibt sie an den externen **Audit-Executor**, der die Befunde bildet; Anfrage, Antwort und Rohantwort werden persistiert. Ist kein Executor konfiguriert oder erreichbar, antwortet der Abruf mit einem eigenen Fehler; ein stilles Ausweichen auf eine interne Bewertung gibt es nicht.

Reports. Der Report-Pfad erzeugt Reports in JSON, CSV oder PDF mit deterministischer Report-ID und Content-Hash; die Erzeugung wird mit Hash, CID und Begründung im Trail festgehalten. Ein ausgehändigter Report ist damit später gegen den Trail verifizierbar.

Workflow-Gate-Läufe als Evidenz. Jeder Gate-Lauf (Kapitel 04) ist selbst ein Compliance-Artefakt mit Schnappschuss-Identität, lokaler Bewertung und Executor-Antwort. Ein Compliance Officer kann einen Lauf im Zustand `REVIEW` mit Begründung entscheiden; die Entscheidung setzt genau den angehaltenen Übergang fort.

9.4 Kontinuierliches Monitoring und Incidents

Der Compliance-Sweep meldet vier Risikoklassen:

- **MISSING_APPROVAL** : Verträge, die auf eine ausstehende erforderliche Freigabe warten (ein Risiko je Vertrag-Freigeber-Paar).
- **CONTRACT_UNDERPERFORMANCE** : laufende Verträge, für die das Zielsystem über den Deployment-Callback eine Regelverletzung gemeldet hat (9.5). Die Verletzung wird als Alert gemeldet, nicht als Request-Fehler abgewiesen, denn der Vertrag ist in Kraft und der Verstoß ein zu dokumentierender Fakt.
- **CONTRACT_DEPLOYMENT_FAILED** : Deployments, deren Versand an das designierte Zielsystem gescheitert ist.
- **UNAUTHORIZED_ACCESS** : Verträge mit persistierten Zugriffsverweigerungen aus der Parteiprüfung (Kapitel 04), ein Risiko je Vertrag-Akteur-Paar.

Geplanter Sweep statt Nachfrage (ADR-32). Dieselbe Erkennungslogik läuft zusätzlich planmäßig: Ein eigenes Kommando (`pacmonitor`) führt genau einen Sweep aus und endet. Es öffnet nur die Datenbank und durchläuft nicht die Startprüfungen des Servers, damit das Monitoring genau dann weiterarbeitet, wenn eine Nachbarkomponente gestört ist; Erkennungen gehen in die Outbox, der laufende Server

verankert und verteilt sie. Ein **CronJob** ruft das Kommando in festem Takt auf und ist im Auslieferungszustand aktiv; ihn abzuschalten bedeutet, dass Risiken nur noch auf Nachfrage gefunden werden.

Ein **Risiko-Register** hält eine Zeile je offener Verletzung, geschlüsselt über Vertrag, Risikoklasse und einen Hash des Detailtexts (ein Vertrag kann mehrere Risiken derselben Klasse tragen). Ein Risiko alarmiert bei der Ersterkennung und erneut, wenn es nach einer Behebung wiederkehrt, nie auf den Sweeps dazwischen. Ein erkanntes Risiko ist **kein Job-Fehlschlag**: Der Sweep endet erfolgreich; ein Fehlschlag bedeutet, dass der Sweep nicht laufen konnte. Jeder Sweep wird selbst als Ereignis verankert, jedes Risiko zusätzlich in der Audit-Kette des betroffenen Vertrags. Die Antwort des Abrufs auf Anfrage bleibt vollständig; die Deduplikation steuert nur das Alarmieren. Wo Sekundenlatenz gefordert ist, ist der Weg ein Event-Bus-Abonnent neben dem Sweep, kein engerer Takt.

Alerts an externe Subscriber. Die Webhook-Plattform liefert neben den Lebenszyklus-Ereignissen `compliance.risk_detected` bei der Ersterkennung eines Risikos aus; Zustellungen sind protokolliert und quittierbar (Anhang A).

Incident-Reports. Befunde außerhalb des automatischen Monitorings reicht der Compliance Officer als Incident-Report ein: Findings aus Risikoklasse und Beschreibung, verknüpft mit der DID der betroffenen Ressource. Jedes Finding wird als eigenes Ereignis in deren Audit-Kette verankert.

Startup-Attestierung der Konfiguration. Beim Start hasht das Backend die sicherheitskritischen gemounteten Konfigurationsartefakte (DID-Dokument, Issuer-Trust-Dokument, Zertifikats-Vertrauensanker, Autorisierungs-Policy) und verankert die Hashes als Attestierungsereignis im Trail. Pinnt der Betreiber erwartete Hashes, bricht eine fehlende oder abweichende gepinnte Datei den Start ab, ebenso eine syntaktisch fehlerhafte Pin-Liste. Die Policy wird mit attestiert, weil sie über dem Trust-Dokument rangiert (Kapitel 05).

Löschnachweise. Die Vernichtung der Inhaltsschlüssel eines Vertrags erzeugt auf jeder beteiligten Instanz ein Ereignis mit Akteur, Vertrag, Bereich und Grund, nie mit Inhalt, abgelegt unter dem instanzweiten Schlüssel. Der Fortschritt der Löschkette über die Instanzgrenze ist über eine eigene Statusabfrage sichtbar.

9.5 Policy-Enforcement: ODRL auf OPA

Vertragsbedingungen sind ODRL 2.0 unter einem DCS-Profil (ADR-6), ausgewertet durch Open Policy Agent, eingebettet als Bibliothek (ADR-11): kein Sidecar, keine zusätzliche Latenz auf dem Freigabe- und Signaturpfad. Das Rego-Modul beantwortet die Wahrheit **einer** Bedingung; die deontische Logik darüber (Verbote, `and` / `or` / `xone`, Pflichten, Operanden der Nutzungszeit) liegt im auswertenden Code.

Das Enforcement ist zwischen DCS und Zielsystem geteilt (ADR-33):

Moment	Wer entscheidet	Wirkung
Vor der Signatur (Freigabe, Signaturvorbereitung)	Das DCS wertet die im Vertrag inline getragenen Feldwerte gegen die eigenen Bedingungen aus	Serverseitige Ablehnung constraint-verletzender Werte; Findings im Trail
Ausführung / KPI-Monitoring	Das Zielsystem, das die Regeln vollstreckt; das DCS zeichnet dessen Urteil auf	Gemeldete Urteile speisen das Risiko-Register und die Audit-Kette des Vertrags

Das Zielsystem klassifiziert, das DCS protokolliert (ADR-33). Bedingungen, deren Eingaben erst zur Nutzungszeit entstehen (Zeitpunkt, Ort, Menge, erbrachte Leistung), kann nur der vollstreckende Enforcer entscheiden: Das DCS hält die Bedingungen, das Zielsystem die Ereignisse. Eine KPI-Rückmeldung trägt deshalb neben Messwert und Zeitpunkt das Urteil des Zielsystems (`satisfied` , `violated` , `not_evaluated`) und die `@id` der ODRL-Regel, auf die es sich bezieht, wörtlich aus der Policy des Deployment-Umschlags. Das DCS prüft die Zurechenbarkeit und zeichnet auf, statt nachzurechnen: Ein Urteil zu einer Regel, die dieser Vertrag nie ausgerollt hat, wird als Client-Fehler abgewiesen; eine Rückmeldung ohne Urteil wird als `not_evaluated` festgehalten, nie als Erfüllung. Vor der Signatur erscheinen solche Bedingungen als zurückgestellter Befund, nicht als bestandene Prüfung; ein Zielsystem, das nichts meldet, lässt den Vertrag unbeobachtet, und genau das sagt das DCS.

Ergänzend definieren zwei versionierte **Policy-Sets** unter `docs/policies/` die instanzweiten Prüfregelein (Template- und Contract-Content-Set). Jede Regel trägt eine stabile Rule-ID und einen Schweregrad; jedes Finding wird als Audit-Ereignis festgehalten, und ein Befund der Schwere „error“ blockiert den zugehörigen Workflow-Gate-Übergang (Kapitel 04). Diese Regeln prüfen **Struktur und Vorhandensein** deklarierter Felder, nicht deren Werte; Wertgrenzen werden dort durchgesetzt, wo sie als SHACL formuliert sind (Kapitel 03).

9.6 Schnittstellen

Endpunkt	Zweck	Rollen
POST /pac/audit	Audit-Trail eines Scopes abrufen und extern bewerten lassen (Begründung Pflicht, Abruf wird auditiert)	Auditor, Archive Manager
GET /pac/report	Audit-Report erzeugen (JSON/CSV/PDF; Hash und CID im Trail)	Auditor, Archive Manager
POST /pac/report	Incident-Report: Non-Compliance-Findings zu einer Ressource erfassen	Compliance Officer
GET /pac/monitor	Compliance-Sweep auf Anfrage; auch ein Lauf ohne Befund wird auditiert	Compliance Officer
GET /pac/audit/checkpoint/head	Neuester Checkpoint-Head (nur Hashes, Zählwerte, Zeitstempel)	Auditor, Archive Manager, Sys. Auditor
GET /pac/audit/checkpoint/{seq}	Ein bestimmter Checkpoint	Auditor, Archive Manager, Sys. Auditor
GET /pac/audit/checkpoint/proof/{entry_cid}	Merkle-Inklusionsbeweis für einen verankerten Eintrag	Auditor
GET /pac/workflow-gates/{run_id}	Persistierten Workflow-Gate-Lauf lesen	Compliance Officer
POST /pac/workflow-gates/review	Entscheidung zu einem Lauf im Zustand REVIEW festhalten	Compliance Officer
GET /archive/erasure-status	Stand der Schlüsselvernichtung eines Vertrags, lokal und je Peer	Archive Manager, Auditor

9.7 Betriebsverhalten

- **Transiente Anker-Fehler** werden pro Ereignis gezählt und im nächsten Tick erneut versucht; der Eintrag wandert in den nächsten Checkpoint. Sein fachlicher Zeitstempel bleibt unverändert.
- **Dead-Lettering:** Nach 50 fehlgeschlagenen Verankerungsversuchen wird ein Ereignis totgelegt und deutlich geloggt. Dead-Letter-Ereignisse sind Lücken im Trail und brauchen einen Operator; sie sind in der Outbox-Tabelle an ihrer Markierung samt letztem Fehler auffindbar.
- **TSA-Ausfall:** Checkpoints werden ohne Zeitstempel verankert und nachgestempelt; im Head ist der Zustand sichtbar.
- **Volumen-Leck:** Publierte Heads verraten Stapelgröße und Takt, also Aktivitätsvolumen. Das ist eine bewusste Abwägung (ADR-16).
- **Lese-Grenzen:** Ein Voll-Trail-Read läuft maximal 500 Checkpoints rückwärts; komponentenweite Audits lesen die Ressourcen-Ketten parallel.
- **Gelöschte Verträge im Audit:** Die Kette bleibt vorhanden und beweisbar, ihre Rümpfe sind unlesbar; die Audit-Kette dokumentiert die Löschung weiterhin.

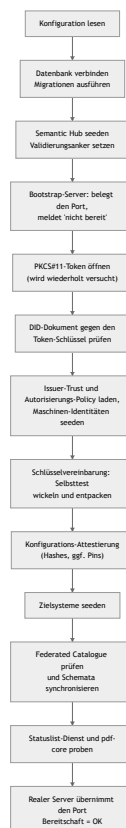
10 Betrieb und Monitoring

Dieses Kapitel beschreibt, wie sich eine DCS-Instanz im Betrieb verhält: was beim Start passiert, welche Hintergrundprozesse laufen, welche Signale es gibt und woran der Betreiber einen Fehlerfall erkennt.

Es enthält bewusst keine Befehle, Werte oder Prozeduren. Installation, Konfigurationsreferenz, Umgebungsmatrix, Upgrade und Rollback stehen im Deployment-Leitfaden; Sicherung und Wiederherstellung im Backup-Leitfaden.

10.1 Startverhalten: fail fast, aber geduldig an einer Stelle

Das Backend prüft seine harten Abhängigkeiten selbst und läuft nie degradiert weiter:



Die Hintergrundprozesse (10.3) starten schrittweise während dieser Initialisierung, nicht an einem einzelnen Punkt; bereit meldet sich die Instanz erst nach der letzten Stufe. Wesentliche Eigenschaften:

- **Der Port ist von Anfang an belegt.** Ein Bootstrap-Server beantwortet den Bereitschaftsendpunkt mit „nicht bereit“, solange die Initialisierung läuft. Kubernetes unterscheidet so einen startenden Prozess von einem betriebsbereiten DCS.
- **Genau eine Stufe wartet statt abubrechen:** das Öffnen des PKCS#11-Tokens, weil bei einer frischen Installation die Token-Provisionierung noch laufen kann. Alles danach ist hart.
- **Passt das DID-Dokument nicht zum Token-Schlüssel, startet die Instanz nicht.** Das ist die häufigste Folge einer Neu-Provisionierung des Tokens ohne Neuerzeugung des Dokuments.
- **Der Selbsttest der Schlüsselvereinbarung ist entscheidungsrelevant:** Schlägt er fehl, wäre kein gespeichertes Artefakt lesbar, und die Instanz startet nicht (Kapitel 07).

- **Ein konfigurierter Federated Catalogue ist eine harte Abhängigkeit:** unvollständig konfiguriert oder nicht funktional erreichbar heißt kein Start; gar nicht konfiguriert heißt Betrieb ohne Katalogfunktionen.
- **Statuslist-Dienst und pdf-core werden per HTTP geprobt,** mit Wiederholungen über ein begrenztes Fenster; bleiben sie unerreichbar, beendet sich der Prozess mit klarer Meldung.
- **Der Audit- und der Workflow-Gate-Executor sind Pflicht:** Fehlt ihre Adresse, startet die Instanz nicht. Sie säßen sonst als fail-closed-Gates in jedem Lebenszyklusübergang (Kapitel 04 und 09).
- **Ungültige Deklarationen brechen den Start ab,** statt still zu wirken: eine unbekannte Rolle im Maschinen-Identitäten-Seed, eine fehlerhafte Zielsystem-Deklaration, eine nicht übersetzbare Autorisierungs-Policy, eine syntaktisch falsche Pin-Liste, ein Trust-Eintrag mit nicht auflösbarem Mechanismus.

10.2 Probes und Bereitschaft

Signal	Bedeutung
Bereitschaftsendpunkt antwortet mit „nicht bereit“	Der Prozess läuft, die Initialisierung ist nicht abgeschlossen. Normal während Installation und Neustart; dauerhaft ist es ein Symptom (10.6)
Bereitschaftsendpunkt antwortet mit OK	Alle Startprüfungen sind durchlaufen, der reale Server bedient die API
Prozess beendet sich mit Fehlermeldung	Eine harte Startabhängigkeit fehlt oder ist falsch. Die Meldung benennt sie; der Orchestrator startet den Container neu

Der Backend-Container wird ausschließlich auf Bereitschaft geprüft; ein Liveness-Signal wird nicht abgefragt. Ein Prozess, der zwar läuft, aber nicht mehr arbeitet, wird deshalb nie automatisch neu gestartet; antwortet er weiter auf dem Bereitschaftsendpunkt, nimmt er sogar Anfragen entgegen. Das folgt aus dem Bootstrap-Verhalten: Eine Lebendigkeitsprobe gegen denselben Endpunkt würde einen Prozess töten, der korrekt auf eine noch nicht provisionierte Abhängigkeit wartet. Wer eine Lebendigkeitsprobe will, fügt sie bewusst hinzu und muss den Startfall abdecken.

pdf-core hat sowohl Bereitschafts- als auch Lebendigkeitsprüfung; die mitgelieferten Begleitdienste bringen ihre eigenen Probes mit.

10.3 Was dauerhaft im Hintergrund läuft

Diese Schleifen zu kennen erklärt fast jedes Verhalten, das von außen „verzögert“ aussieht.

Prozess	Aufgabe	Verhalten bei Störung
Outbox-Publikation	Ereignisse auf den Event-Bus stellen	Hoher Takt; ein nicht erreichbarer Bus verzögert Abonnenten, nicht die Fachlogik
Outbox-Verankerung	Audit-Einträge schreiben, Merkle-Checkpoints bilden, Roots zeitstempeln	Zählt Fehlversuche je Ereignis; nach 50 Versuchen Dead-Letter mit deutlichem Log
Zeitstempel-Nachlauf	Checkpoints ohne Zeitstempel nachträglich stempeln	Läuft, bis die TSA wieder antwortet
PDF-Regenerierung	Auf Lifecycle-Ereignisse hin rendern/amendieren	Ein Ereignis wird höchstens einmal zugestellt; Fehlschläge fängt der Retry-Sweep
Regenerations-Retry-Sweep	Entitäten finden, deren gespeichertes PDF nicht dem aktuellen Dokument entspricht	Begrenzte Stapel, begrenzte Versuche, wachsender Abstand
Föderations-Versand	Vertrags-PDFs an Peers ausliefern	Fehlschläge landen in der Retry-Tabelle; ein Scheduler versucht sie erneut
Löschungs-Retry	Offene Peer-Löschanforderungen erneut zustellen	Eigene Warteschlange, gleicher Takt wie der Föderations-Retry
Statuslisten-Publikation	Widerrufseinträge der Lifecycle-Credentials veröffentlichen	Eigener Arbeiter, unabhängig vom Request-Pfad
Ablauf-Job	Verträge nach Ablaufdatum auf <code>EXPIRED</code> setzen	Loggt Fehler je Vertrag und macht weiter; ohne Expiration-Policy wird übersprungen (Kapitel 04)
Webhook-Zustellung	Ereignisse an registrierte Abonnenten liefern	Zustellungen werden protokolliert und sind quittierbar
Compliance-Sweep (CronJob)	Compliance-Risiken planmäßig erkennen (ADR-32)	Eigenes Kommando, eigener Pod; ein erkanntes Risiko ist kein Job-Fehlschlag

Der Compliance-Sweep ist bewusst ein eigener geplanter Job: Ein fehlgeschlagener Lauf ist ein sichtbares Objekt im Orchestrator, der Takt ist ohne Neustart änderbar, und er läuft weiter, wenn eine Nachbarkomponente gestört ist (Kapitel 09).

10.4 Metriken

Das Backend exponiert einen Prometheus-Endpunkt mit zwei HTTP-Basismetriken: Anzahl und Dauer der Requests nach Methode, Pfad und Statuscode. Damit sind Fehlerraten, Latenzverteilungen und Lastprofile pro Endpunkt auswertbar (Anteil 429 aus dem Anfragebudget, Anteil 5xx, Latenz der Export- und Signatur-Pfade). Der Endpunkt liegt außerhalb der authentifizierten API und des Anfragebudgets; das Einsammeln richtet der Betreiber in seiner eigenen Monitoring-Infrastruktur ein.

Anwendungsfachliche Metriken (verankerte Checkpoints, Länge der Retry-Warteschlangen, offene Risiken) exportiert das Backend **nicht** als Zeitreihen; diese Beobachtungen laufen über Logs, Compliance-API und Datenbank (10.5).

10.5 Was der Betreiber sieht

Logs, auf die es ankommt:

- jeder verankerte Checkpoint mit Sequenz, Eintragszahl, Root und Zeitstempel-Status; bleiben diese Meldungen aus, während das System Ereignisse produziert, steht die Verankerung
- jedes dead-letterte Audit-Ereignis, unübersehbar formuliert; es ist der einzige Weg, auf dem der Trail eine Lücke bekommt, und braucht einen Operator
- Wartemeldungen des Token-Öffnens beim Start
- Wiederholversuche des Föderations-Versands und des Regenerations-Sweeps; nachgeholte TSA-Zeitstempel
- die Diagnose bei einem Inhalts-Mismatch eines empfangenen Peer-PDFs

Zustände in der Datenbank:

Bereich	Frage, die er beantwortet
Outbox mit Dead-Letter-Markierung	Gibt es Ereignisse, die nie in den Trail gelangt sind, und warum?
Retry-Tabellen für Föderationsversand und Peer-Löschung	Welche Sendungen bzw. Löschanforderungen hängen, seit wann und mit welchem Fehler?
Risiko-Register des Compliance-Monitorings	Welche Verstöße sind offen, seit wann, welche wurden geschlossen?

API-Sichten: Checkpoint-Head (fehlender Zeitstempel zeigt eine TSA-Störung), Compliance-Sweep auf Anfrage, Workflow-Gate-Läufe (warum ein Übergang blockiert wurde oder wartet), Erasure-Status, Schlüsselinventar, Webhook-Zustellprotokoll.

Ereignisse für externe Systeme: Registrierte Abonnenten erhalten benannte Lebenszyklus-Ereignisse und den Compliance-Alarm bei Ersterkennung eines Risikos (Anhang A). **Subscriptions und Zustellprotokoll liegen im Prozessspeicher:** Sie überleben keinen Neustart und werden zwischen Replikaten nicht geteilt. Wer dauerhafte Zustellung braucht, registriert nach jedem Neustart neu.

10.6 Fehlerbilder

Die folgenden Bilder sind aus dem Verhalten der Artefakte abgeleitet bzw. im Betrieb der Demonstrationsumgebung beobachtet. Konkrete Behebungsschritte stehen im Deployment-Leitfaden.

Symptom	Ursache	Richtung der Behebung
Pod läuft, Bereitschaft bleibt dauerhaft negativ, Log wiederholt Wartemeldungen zum PKCS#11-Token	Das Token ist nicht provisioniert oder nicht gemountet	Provisionierungs-Job und Token-Volume prüfen
Prozess beendet sich sofort mit einer Meldung zum DID-Dokument	Das DID-Dokument passt nicht zum Schlüssel im Token, typisch nach Neu-Provisionierung ohne Neuerzeugung des Dokuments	DID-Dokument aus dem Token neu ableiten
Prozess beendet sich mit einer Meldung zur Schlüsselvereinbarung	Publizierter keyAgreement -Schlüssel und Token-Schlüssel passen nicht zusammen, oder das Token kann die Ableitung nicht	Dokument und Token abgleichen; bei einem produktiven HSM die Ableitungsfähigkeit für die Kurve prüfen
Prozess beendet sich mit einer Meldung zum Federated Catalogue	Katalog unvollständig konfiguriert oder nicht funktional erreichbar	Katalog-Konfiguration vervollständigen oder Katalog abschalten
Prozess beendet sich mit einer Meldung zu einer gepinnten Konfigurationsdatei	Eine attestierte Datei fehlt oder weicht vom Pin ab; oder die Pin-Liste ist syntaktisch falsch	Datei oder Pin korrigieren (Kapitel 09)
Jeder Lebenszyklusübergang wird blockiert, obwohl der Vertrag korrekt ist	Der Workflow-Gate-Executor ist nicht erreichbar; das Gate ist fail-closed	Executor-Flow prüfen; der Gate-Lauf nennt die Ursache
Ein Audit-Abruf antwortet mit einem Executor-Fehler	Der Audit-Executor ist nicht erreichbar oder antwortet nicht verwertbar	Executor-Flow prüfen
Ein Vertrag ist nicht exportierbar und wird nicht an den Peer verschickt	Die PDF-Regeneration ist fehlgeschlagen	Der Retry-Sweep holt es nach; die Ursache steht im Log des Regenerators
Export antwortet mit „wird gerade regeneriert“	Eine Regeneration lief länger als das Wartefenster des Exports	Erneut abrufen; anhaltend deutet es auf eine pdf-core- oder Speicherstörung
Verifikation meldet „Artefakt nicht authentisch“	Die gespeicherten Bytes sind nicht die geschriebenen: Manipulation oder Substitution im Speicher	Als Sicherheitsvorfall behandeln; die Merkle-Beweise bleiben gültig (Kapitel 07, 09)
Export, Verifikation oder Bundle antworten „Inhalt gelöscht“	Die Inhaltsschlüssel des Vertrags wurden vernichtet	Erwartetes Verhalten nach einer Löschung; Erasure-Status zeigt Zeitpunkt und Akteur
Peer-Versand hängt in der Retry-Tabelle	Peer nicht erreichbar, DID-Auflösung scheitert oder das Agreement Credential wird abgelehnt	Peer-Erreichbarkeit und DID-Auflösung prüfen; Details im Log und im Retry-Eintrag
Föderation ist abgeschaltet, obwohl konfiguriert	Der Trust-PDP ist nicht gesetzt oder nicht erreichbar; das Gate ist fail-closed	Policy-Endpunkt prüfen; jede Ablehnung liegt als Incident im Trail

Symptom	Ursache	Richtung der Behebung
Eine Sendung einer Gegenseite wird abgelehnt, obwohl der Peer bekannt ist	Der mitgeschickte Vollmachtsnachweis verifiziert nicht: Aussteller nicht für <code>peer</code> zugelassen, Organisation nicht zugestanden, Credential abgelaufen oder widerrufen	Issuer-Vertrauen der empfangenden Instanz prüfen (Kapitel 05, 08)
Ein Wallet-Login schlägt reproduzierbar fehl	Aussteller nicht für <code>login</code> zugelassen, Organisation nicht zugestanden, Statusliste nicht erreichbar, oder Zertifikatskette ohne konfigurierte Vertrauensanker	Trust-Konfiguration prüfen; der Fehler benennt die Stufe
Antworten mit <code>429</code>	Das Anfragebudget je Credential ist ausgeschöpft	Aufrufverhalten prüfen; das Budget ist konfigurierbar
Webhook-Abonnenten erhalten nach einem Neustart nichts mehr	Subscriptions liegen im Prozessspeicher	Neu registrieren
Signatur-Einreichung wird abgelehnt, obwohl das Dokument gültig signiert ist	Kein DSS konfiguriert oder nicht erreichbar; oder das Byte-Prefix stimmt nicht, weil ein anderes als das vorbereitete Dokument signiert wurde	DSS prüfen; der Signatar muss exakt das vorbereitete Dokument signieren (Kapitel 06)
Externe PDF-Prüfprogramme melden „nach der Signatur geändert“	Der Provenance-Re-Anchor nach der Signatur (ADR-26)	Erwartetes Verhalten; die eigene Verifikation weist genau diese eine Revision nach (Kapitel 07)

10.7 Kapazitäts- und Betriebsgrenzen, die man kennen muss

- **Ein Replikat pro Instanz.** Die Webhook-Subscriptions liegen im Prozessspeicher; mehrere Replikate hätten unterschiedliche Sichten. Die Skalierung liegt in der Vertikalen.
- **Die Verankerung ist strikt sequenziell** und hängt an TSA- und Speicher-Roundtrips; große Rückstände arbeiten sich in Stapeln ab.
- **Ein Voll-Trail-Read ist begrenzt** (maximal 500 Checkpoints rückwärts), damit ein Audit-Abruf innerhalb seiner Frist bleibt.
- **Sicherungen verzögern die Vollendung einer Löschung.** Ein heute vernichteter Schlüssel existiert in der gestrigen Sicherung weiter; nach einer Wiederherstellung müssen die seither erfolgten Löschungen nachgezogen werden (Backup-Leitfaden).
- **Die Schlüsselvernichtung entfernt keine Chiffre aus dem Speicher;** entfernt werden die Archiv-Snapshots (Kapitel 07).

Anhang A Schnittstellenreferenz

Dieser Anhang listet die Schnittstellen einer DCS-Instanz auf drei Ebenen: die HTTP-API-Gruppen des Backends, die Ereignisse auf dem internen Event-Bus und die öffentlichen well-known-Endpunkte. Er beschreibt Gruppen und Zweck, nicht einzelne Parameter; die vollständige, maschinenlesbare API-Beschreibung liefert jede laufende Instanz selbst (Swagger-Oberfläche und OpenAPI-3-Spezifikation).

Alle fachlichen APIs sind, sofern nicht anders vermerkt, durch das OIDC-Bearer-Token geschützt; die Scopes im Access Token entsprechen den Rollen-Claims des Nutzers. Maschinelle Aufrufer authentifizieren sich mit Client-Credentials-Tokens ihrer eigenen OAuth2-Clients; ihre Rollen liest das Backend anhand der Client-ID aus der Registry, nie aus Token-Claims. Öffentlich sind die well-known-Endpunkte, der C2PA-Manifestabruf, die Semantic-Hub-Auflösung sowie die von Wallets und Peer-Instanzen aufgerufenen Callback-Pfade.

A.1 HTTP-API-Gruppen

Gruppe	Basispfad	Zweck
Auth	/auth/...	Endnutzer-Login: OIDC-Session-Management und OpenID4VP-Presentation (inkl. PID-Presentation)
TemplateRepository	/template/...	Lebenszyklus der Vertragsvorlagen
SemanticHub	/semantic/...	Versionierte JSON-LD-Kontexte, SHACL-Shapes, Ontologien, Validierungsprofile, Klausel-Katalog
TemplateCatalogueIntegration	/catalogue/template/...	Lesende Anbindung an den XFSC Federated Catalogue
ContractWorkflowEngine	/contract/..., /machine-identities/...	Vertragslebenszyklus; Zielsystem-Registry und Maschinen-Identitäten
SignatureManagement	/signature/...	Signatur-Ceremonies, Prüfung, Provenance, Compliance
PDFGeneration	/pdf/..., /contract/export/..., /template/export/...	Export und unabhängige Verifikation der PDF-Repräsentationen
ContractStorageArchive	/archive/...	Langzeitarchiv abgeschlossener Verträge, Löschung und Löschststatus
ProcessAuditAndCompliance	/pac/...	Audit-Abruf, Reports, Monitoring, Merkle-Checkpoints, Workflow-Gate-Läufe
DcsToDcs	/peer/...	Föderation: PDF-/JAdES-Austausch und Löschanforderung zwischen DCS-Instanzen
KeyInventory	/admin/hsm-keys	Lesendes Inventar der HSM-Schlüssel (Sys. Administrator)
DIDService	/.well-known/...	DID-Dokument, Agreement Credential, Föderationsregeln (öffentlich)
C2PAService	/c2pa/...	Öffentlicher Abruf des C2PA-Manifests eines Vertrags
Webhook-Plattform	/orice/...	Abonnements, Zustellungen und Callbacks für externe Ereignisempfänger

Auth (/auth/...)

Zwei Ablauffamilien: der OIDC-Login mit OpenID4VP-Presentation und die davon unabhängige, einmalige PID-Presentation ohne Session.

Endpunktgruppe	Endpunkte	Zweck
Login und OIDC-Session	POST /auth/login, /auth/login/renew, /auth/login/challenge, GET /auth/login/status; GET /auth/consent, /auth/callback, POST /auth/refresh, GET /auth/logout, /auth/logout-complete	OID4VP-Login starten, verlängern und an die Hydra-Challenge binden; Status-Polling; Consent und Callback; Token-Refresh über HttpOnly-Cookie; Abmeldung
Wallet- und PID-Presentation	GET/POST /auth/presentation/request/{state}, POST /auth/presentation/callback; gleiche Mechanik sessionlos unter /auth/pid/presentation/...	Signiertes Authorization-Request-Objekt (Request-by-Reference) und Direct-Post-Rückkanal; die PID-Variante ohne Session (Kapitel 05)

TemplateRepository (/template/...)

Endpunktgruppe	Endpunkte	Zweck
Anlage und Pflege	POST /template/create, POST /template/copy, PUT /template/update, POST /template/update_manage	Vorlage anlegen, kopieren, inhaltlich bzw. verwaltend ändern
Freigabeprozess	POST /template/submit, POST /template/verify, POST /template/approve, POST /template/reject	Zur Prüfung einreichen, semantisch verifizieren, freigeben oder ablehnen
Abruf und Suche	GET /template/search, GET /template/retrieve, GET /template/retrieve/{did}, GET /template/{did}, GET /template/history/{did}	Vorlagen suchen, listen, per DID auflösen, Historie einsehen
Provenance und Audit	GET /template/provenance/{did}, GET /template/audit	Herkunftsnachweis und Audit-Einträge einer Vorlage
Verteilung	POST /template/register, POST /template/publish, POST /template/archive	Version registrieren, im Federated Catalogue veröffentlichen, ausmustern

SemanticHub (/semantic/...)

Sämtliche lesenden Endpunkte dieser Gruppe sind öffentlich: Erzeugte Artefakte tragen Hub-Anker, die externe Prüfer ohne DCS-Login auflösen können müssen. Schreibend ist nur die Schema-Verwaltung (Template Manager).

Endpunktgruppe	Endpunkte	Zweck
Schema-Verwaltung	POST /semantic/schema/register , POST /semantic/schema/rollback	Neue Schema-Version registrieren, auf eine frühere zurücksetzen
Schema-Abruf	GET /semantic/schema/retrieve , GET /semantic/schema/versions , GET /semantic/schema/list	Aktive oder benannte Version abrufen, Versions- und Schemaliste
Artefakt-Auflösung	GET /semantic/context/{name} , GET /semantic/shapes/{name} , GET /semantic/ontology/{name} , GET /semantic/profile/{name}	Kontext, Shapes, RDF-Konfiguration und Validierungsprofil unter stabilen Ankern
Klausel-Katalog	GET /semantic/clauses	Klauseltypen für die Palette des Vorlagen-Builders

TemplateCatalogueIntegration (/catalogue/template/...)

Endpunkte	Zweck
GET /catalogue/template/retrieve , GET /catalogue/template/retrieve/{did} , GET /catalogue/template/search	Im Federated Catalogue veröffentlichte Vorlagen listen, per DID abrufen und durchsuchen (menschliche Vertragsrollen)

ContractWorkflowEngine (/contract/... , /machine-identities/...)

Endpunktgruppe	Endpunkte	Zweck
Anlage und Pflege	POST /contract/create , PUT /contract/update , POST /contract/renew	Vertrag aus Vorlage anlegen, im Entwurf ändern, Folgevertrag ableiten
Angebot	POST /contract/offer , POST /contract/withdraw , POST /contract/submit	Der Gegenpartei anbieten, vor Freigabe zurückziehen, Zustandsübergang einreichen
Verhandlung	POST /contract/negotiate , PUT /contract/negotiation_draft , GET/DELETE /contract/negotiation_draft/{did} , POST /contract/respond	Change Requests stellen, Verhandlungsentwurf zwischenspeichern, Gegenvorschlag annehmen/ablehnen
Entscheidung	POST /contract/approve , POST /contract/reject , GET /contract/review	Freigeben oder ablehnen, Review-Sicht der offenen Aufgaben
Abruf und Suche	GET /contract/retrieve , GET /contract/retrieve/{did} , GET /contract/{did} , GET /contract/history/{did} , GET /contract/search , GET /contract/templates	Verträge listen, per DID auflösen (parteibeschränkt), Historie, Suche, verfügbare Vorlagen
Abschluss und Betrieb	POST /contract/store , POST /contract/terminate , GET /contract/kpis/{did} , POST /contract/audit	Evidenz sichern, beenden, KPI-Beobachtungen einsehen, Audit-Auszug
Deployment	POST /contract/deploy , POST /contract/target/designate , POST /contract/deployment/callback	An das designierte (oder ein explizit gewähltes) Zielsystem ausrollen; Designation setzen oder löschen; Rückmeldung des Zielsystems (Ack/Status, KPI-Beobachtungen samt Urteil und Regelbezug, ADR-33), authentifiziert als dessen eigener OAuth2-Client und nur für Deployments an genau dieses Ziel
Zielsystem-Registry	GET/POST/PUT/DELETE /contract/targets , POST /contract/targets/{id}/credential	Registry administrieren (Schreibzugriff Sys. Administrator und Integration Manager, Lesen zusätzlich Contract Manager); Löschen wird verweigert, solange ein Vertrag das Ziel designiert; Callback-Credential ausstellen/rotieren (Secret genau einmal sichtbar)
Maschinen-Identitäten	GET/POST /machine-identities , PUT/DELETE /machine-identities/{id} , POST /machine-identities/{id}/credential	Registry verwalten: Anlegen provisioniert den OAuth2-Client und liefert das Secret genau einmal; Deaktivieren weist Aufrufe sofort ab; Löschen entfernt auch den Client; Rotation invalidiert das alte Secret unmittelbar (Sys. Administrator)

SignatureManagement (/signature/...)

Endpunktgruppe	Endpunkte	Zweck
Wallet-Ceremony	POST /signature/request , GET /signature/request/{ceremony_id} , POST /signature/request/{ceremony_id}/publish , GET/POST /signature/request/{ceremony_id}/object , GET /signature/request/{ceremony_id}/document , GET /signature/request/{ceremony_id}/payload , POST /signature/request/{ceremony_id}/callback	Ceremony starten, Status abfragen, Request-Objekt, zu signierendes PDF und kanonische JSON-LD-Payload an die Wallet ausliefern, Ergebnis über den nonce-gebundenen Callback entgegennehmen
Direkter Signaturfluss	POST /signature/prepare , POST /signature/submit	Vorbereitetes, byte-gepinntes PDF ausliefern und fertige externe Signatur einreichen
Prüfung	POST /signature/verify , POST /signature/validate , POST /signature/compliance	Integrität und Provenance verifizieren, DSS-gestützte Validierung, Niveau-Prüfung
Abruf und Nachweis	GET /signature/retrieve , GET /signature/retrieve/{did} , GET /signature/provenance/{did} , GET /signature/view , GET /signature/audit	Signierliste, Signaturumschlag je Vertrag, Provenance-Kette, Compliance-Viewer, Audit-Einträge
Verwaltung	POST /signature/revoke	Signatur widerrufen

PDFGeneration (/pdf/... , Export-Bundles)

Endpunktgruppe	Endpunkte	Zweck
PDF-Export	GET /pdf/export/contract/{did} , GET /pdf/export/template/{did}	Deterministisch gerendertes PDF eines Vertrags bzw. einer Vorlage
Bundle-Export	GET /contract/export/{did} , GET /template/export/{did}	ZIP-Bundle mit PDF und maschinenlesbaren Artefakten
Verifikation	GET /pdf/verify/contract/{did} , GET /pdf/verify/template/{did}	Unabhängiger Neurender-Abgleich mit benannter Fehlerklasse

ContractStorageArchive (/archive/...)

Endpunkte	Zweck
POST /archive/store	Abgeschlossenen Vertrag mit Evidenz archivieren
GET /archive/retrieve , GET /archive/search	Archiveinträge abrufen und strukturiert durchsuchen (Volltext, Name, Zustand, Partei, Gültigkeitszeitraum, Annotations-Tag)
POST /archive/annotate	Eintrag mit Zusammenfassung/Tags versehen
DELETE /archive/delete	Eintrag mit Begründung löschen: Soft-Delete, Entfernen der Snapshots und Vernichtung der Inhaltsschlüssel auf beiden Instanzen (ADR-28)
GET /archive/erasure-status	Stand der Schlüsselvernichtung: lokale Zerstörungsnachweise und offene Peer-Anforderungen
GET /archive/statistics	Archiv-Dashboard: Bestands- und Speicherstatistik, letzte Aktionen, demnächst auslaufende Verträge, Evidenz-Vollständigkeit
GET /archive/audit	Audit-Einträge des Archivs

ProcessAuditAndCompliance (/pac/...)

Endpunkte	Zweck
POST /pac/audit	Audit-Abruf: Evidenz sammeln und extern bewerten lassen (Begründung Pflicht, Abruf wird auditiert)
GET /pac/report	Audit-Report erzeugen (JSON/CSV/PDF)
POST /pac/report	Incident-Report zu einer Ressource erfassen
GET /pac/monitor	Compliance-Sweep auf Anfrage; auch ein Lauf ohne Befund wird auditiert
GET /pac/audit/checkpoint/head , GET /pac/audit/checkpoint/{seq} , GET /pac/audit/checkpoint/proof/{entry_cid}	Aktueller Merkle-Checkpoint-Kopf, ein bestimmter Checkpoint, Inklusionsbeweis für einen Audit-Eintrag
GET /pac/workflow-gates/{run_id} , POST /pac/workflow-gates/review	Workflow-Gate-Lauf lesen; eine REVIEW - Entscheidung mit Begründung festhalten

DcsToDcs (/peer/...)

Föderationsschnittstelle zwischen zwei DCS-Instanzen. Alle Endpunkte stehen unter dem Trust Gate (Agreement Credential plus lokaler Policy-Endpunkt, fail-closed) und werden per did:web-Challenge-Response im Request-Body authentifiziert, nicht per Session-Token.

Endpunkte	Zweck
POST /peer/contracts/pdf	Eingang eines Vertrags-PDFs mit Zustand, optional JAdES, Vollmachtsnachweis und gewickeltem Inhaltsschlüssel
POST /peer/contracts/erase	Löschanforderung: Vernichtung der Inhaltsschlüssel dieses Vertrags auf der empfangenden Instanz
GET /peer/contracts/provenance	Gespeicherte JAdES-Provenienz eines empfangenen Vertrags (JWT-geschützt, für lokale Nutzer)

KeyInventory (/admin/hsm-keys)

Endpunkt	Zweck
GET /admin/hsm-keys	Lesendes Inventar: Label, Zweck und aktive Version jedes HSM-Schlüssels (Sys. Administrator). Rotation ist ein Betriebsverfahren, keine API-Handlung

C2PAService (/c2pa/...)

Endpunkt	Zweck
GET /c2pa/manifest/{contract_id}	Öffentlicher, nicht authentifizierter Abruf des C2PA-Manifest-Stores eines Vertrags; optional die Kettenaufzählung

A.2 Ereignisse auf dem Event-Bus

Alle Backend-Domänen publizieren ihre Ereignisse als CloudEvents über einen gemeinsamen NATS-Topic, zugestellt nach dem Transactional-Outbox-Muster (Kapitel 09). Konsumenten unterscheiden Ereignisse über den CloudEvent-Typ.

Abonnenten innerhalb der Instanz:

Konsument	Reagiert auf	Wirkung
DCS-to-DCS-Synchronizer	OFFER_CONTRACT , PDF_REGENERATED , APPLIED_SIGNATURE , REVOKE_SIGNATURE	Versand des Vertrags-PDFs an die Peer-Instanz bei versandpflichtigen Zuständen
PDF-/C2PA-Regenerator	Lebenszykluseignisse von Verträgen und Vorlagen	Hintergrund-Neurendering und Anfügen einer C2PA-Lifecycle-Assertion
Webhook-Plattform	Domänenereignisse mit Webhook-Abbildung	Weiterleitung an registrierte externe Abonnenten
Auto-Deployment	APPLIED_SIGNATURE	Anstoß der Zielsystem-Auslieferung über dasselbe Kommando wie der manuelle Deploy
Event-Logger (optional)	alle Ereignisse	Diagnose-Mitschnitt, nur wenn ausdrücklich eingeschaltet

Die CloudEvent-Typen folgen dem Kommando-Vokabular ihrer Domäne (CREATE_CONTRACT , PUBLISH_CONTRACT_TEMPLATE , APPLIED_SIGNATURE , KEY_SHREDDED , PAC_COMPLIANCE_RISK , ...) und sind zugleich der Ereignistyp im Audit-Trail. Maßgeblich ist der Typ auf dem Bus; reine Lesezugriffe erzeugen

Lookup-Ereignisse, die nicht im Trail verankert werden (Kapitel 09).

Webhook-Plattform (/orce/...)

Die Webhook-Plattform übersetzt interne Ereignisse in benannte, abonnierbare Alert-Ereignisse:

`contract.created`, `contract.submitted`, `contract.approved`, `contract.rejected`, `contract.negotiated`,
`contract.terminated`, `contract.expired`, `template.created`, `template.approved`, `template.updated`,
`template.registered`, `template.deprecated`, `compliance.risk_detected`.

Endpunkt	Zweck
GET /orce/events	Katalog der abonnierbaren Ereignisnamen
GET /orce/webhooks, POST /orce/webhooks, DELETE /orce/webhooks/{id}	Subscriptions listen, registrieren (Ereignis, Callback-URL, Secret), löschen
POST /orce/callbacks	Quittung des Abonnenten für eine Zustellung
GET /orce/deliveries	Zustellprotokoll (Ergebnis, Statuscode, Quittungsstatus)

Subscriptions und Zustellprotokoll liegen im Prozessspeicher und überdauern keinen Neustart (Kapitel 10).

A.3 Well-known-Endpunkte

Öffentliche, nicht authentifizierte Endpunkte an der Origin-Wurzel der Instanz; dieselben Dokumente sind zusätzlich unter dem API-Präfix erreichbar. Sie sind die Auflösungsbasis des Föderations-Trust-Modells.

Endpunkt	Inhalt	Zweck
GET /.well-known/did.json	DID-Dokument der Instanz (did:web)	Instanzidentität; öffentliche Schlüssel zu den im HSM gehaltenen privaten Schlüsseln, einschließlich des Schlüsselvereinbarungs-Schlüssels
GET /.well-known/dcs-agreement-credential.json	Selbstsigniertes Agreement Credential	Nachweis der Regelakzeptanz; Prüfgegenstand des Trust Gates auf Ein- und Ausgangspfad
GET /.well-known/dcs-federation-rules.md	Föderationsregeln	Das Regeldokument, auf das sich das Agreement Credential per Hash bezieht

Ergänzend zur Auflösung durch Dritte:

- **OID4VP-Metadaten** werden nicht als eigenes well-known-Dokument ausgeliefert: Die Verifier-Metadaten stecken im signierten Authorization-Request-Objekt, verankert über die Zertifikatskette im JAR-Header und den DNS-gebundenen Client-Identifizier.
- Die **OIDC-Discovery** liefert nicht das DCS-Backend, sondern der mitbetriebene OAuth2-Provider.
- GET /c2pa/manifest/{contract_did} ist das öffentliche Gegenstück für die Provenance-Auflösung eines Vertrags.
- Der **Metrik-Endpunkt** liegt außerhalb der authentifzierten API und außerhalb des Anfragebudgets (Kapitel 10).

Anhang B Entscheidungsarchiv

Verweisliste auf die Architecture Decision Records unter `docs/` . Jeder Eintrag nennt die These des ADRs und, wo relevant, seine Beziehung zu anderen ADRs. Begründungen und Konsequenzen stehen ausschließlich im jeweiligen ADR. ADRs ohne abweichenden Vermerk sind unverändert in Kraft.

Projektreihe

ADR	These und Status
ADR-1	PKCS#11/HSM ist der einzige Key-Custody-Mechanismus für alle DCS-eigenen Signaturschlüssel. In Teilen superseded durch ADR-12: PAdES-Vertragssignaturen zählen nicht zu den DCS-Key-Custody-Touchpoints.
ADR-2	Eine einzige Vertrags-State-Machine mit einer Transitionstabelle. Der ursprünglich implizierte Full-State-Broadcast-Synchronizer ist durch ADR-13 zurückgezogen.
ADR-3	Signatursemantik: Identitätsbindung unter der Signatur, Embed-then-Sign. Die org-key-basierte AES über das PDF ist durch ADR-12 und ADR-20 zurückgezogen.
ADR-4	C2PA-Provenance wird doppelt ausgeliefert: eingebettet als JUMBF im PDF und als standardkonformes Remote-Manifest.
ADR-5	Festlegung je XFSC-Komponente, ob sie übernommen oder substituiert wird; schließt Graph-Store und Deployment-Lifecycle des übernommenen Federated Catalogue ein.
ADR-6	Vertragsbedingungen sind echtes ODRL unter einem DCS-Profil und werden serverseitig durchgesetzt. Nur der Evaluator-Mechanismus ist durch ADR-11 superseded.
ADR-7	Vertragshierarchie-Links zeigen ausschließlich vom Kind zum Elternvertrag, nie umgekehrt.
ADR-8	Enforcement bezieht SHACL-Shapes und Validierungsprofil ausschließlich aus versionsgepinnten Semantic-Hub-Ständen.
ADR-9	Festlegung der SHACL-Engine für die semantische Validierung.
ADR-10	Der Klausel-Katalog wird als vorverdautes JSON zusammen mit dem rohen Turtle in einer Antwort transportiert.
ADR-11	ODRL-Constraints werden auf eingebettetem OPA evaluiert. Supersedet den Evaluator-Mechanismus aus ADR-6; das Ausführungs-/KPI-Moment ist seinerseits durch ADR-33 superseded, das Vor-Signatur-Moment steht unverändert.
ADR-12	Die Vertragssignatur ist wallet-getrieben: Das DCS ist OpenID4VP-Relying-Party und Signaturvalidator und hält keinen Vertragssignaturschlüssel. Supersedet die Organisationssignatur-Klausel von ADR-3 und beschneidet ADR-1; die Akzeptanzpfad-Klauseln sind ihrerseits durch ADR-20 superseded.
ADR-13	DCS-to-DCS-Föderation ist ein Austausch von Vertrags-PDF und JAdES, kein State-Broadcast; jede Instanz führt Workflow und RBAC lokal. Supersedet den Synchronizer aus ADR-2; die dritte Vertrauensschicht ist durch ADR-19 superseded.
ADR-14	Intern soll genau eine JSON-LD-Form existieren, expandiert, mit Expansion nur an den Ingestion-Kanten. Als Entscheidung angenommen, nicht umgesetzt: Die ausgelieferte interne Form ist die kanonische <code>dc:</code> -präfixierte Hülle (Kapitel 03).
ADR-15	Ein verhandelbarer Datenpunkt ist genau ein typisierter, SHACL-abgeleiteter, per <code>@id</code> verlinkter Knoten statt einer mehrstufigen Placeholder-Indirektion. Superseded durch ADR-22, das den Kern behält und den Knoten umbenennt.
ADR-16	Manipulationsnachweis des Audit-Trails entsteht über Merkle-Checkpoints je Verankerungs-Tick mit externer Verankerung. Verfeinert den event-sourced Audit-Trail, dessen Hash-Ketten je Ressource bestehen bleiben.
ADR-17	Eine Maschinen-Identität kann keine AES erzeugen: Die Klasse System Contract Signer erhält keinerlei Signatur-Scope. Um die Siegel-Option erweitert durch ADR-21.

ADR	These und Status
ADR-18	Gegenüber dem Federated Catalogue tritt das DCS-Deployment als ein Participant mit einem Service-Account auf, nicht der einzelne Publisher. Ein entfernter, nicht mitdeployter Katalog erfordert eine explizite Trust-Boundary-Bestätigung, sonst schlägt das Rendering fehl.
ADR-19	Föderationsvertrauen ist geschichtet: did:web-Identität mit Zertifikatskette, Regelakzeptanz per Agreement Credential und ein lokaler Policy-Endpunkt (fail-closed). Ersetzt die dritte Vertrauensschicht von ADR-13.
ADR-20	Härtung des Signatur-Akzeptanzpfads: Nonce-Bindung des Ceremony-Callbacks, Byte-Pinning des zu signierenden Dokuments, atomare Ceremony-Konsumption, level-bewusste Gates, Zertifikat-zu-Identität-Prüfung, JAdES-Validierung. Supersedet die Akzeptanzpfad-Klauseln von ADR-12.
ADR-21	System Contract Sealer: das von ADR-17 zurückgestellte elektronische Siegel über den generischen Verify-on-Reupload-Pfad statt einer Lockerung des AES-Verbots. Als Richtung akzeptiert, nicht implementiert.
ADR-22	Der typisierte Knoten ist ein Feld, kein Placeholder: <code>dcs:contractFields</code> trägt die Felddeklarationen, <code>dcs:contractData</code> typisierte Domänenobjekte mit Feldreferenzen. Supersedet ADR-15; durch ADR-23 amendiert.
ADR-23	Der Contract-Data-Graph ist generisch: beliebige SHACL-Vokabulare aus dem Semantic Hub, Properties als Literal, Feldreferenz oder Objektreferenz, In-Document-Auflösung und Validierung einer feld-materialisierten Kopie. Amendiert ADR-22, dessen Zwei-Ebenen-Form als Spezialfall gültig bleibt.
ADR-24	v1-Signatur-Scope: wallet-basierte AES; das erreichte Level bestimmt der angebundene Vertrauensdienst, nicht die Codebasis. QES und weitere Formate sind als kundenbestätigte Abweichungen dokumentiert; die technischen Konformitäts-Gates bleiben erhalten.
ADR-25	Zielsysteme sind eine persistierte, administrierte Registry, und jeder Vertrag designiert sein eigenes Deployment-Ziel; ein Vertrag ohne Designation deployt nicht. Die Callback-Authentifizierung ist durch ADR-27 superseded.
ADR-26	Die Signatur wird über der Provenance angebracht, und die Provenance danach über der Signatur neu verankert (provenance-only C2PA-Update-Manifest als inkrementelles Update); Signaturvalidität behält in jedem Konflikt Vorrang.
ADR-27	Maschinen-Credentials werden ausgestellt, nicht konfiguriert: Jeder maschinelle Aufrufer erhält einen eigenen, über den OAuth2-Provider provisionierten Client mit genau einmal angezeigtem Secret; die Deployment-Deklaration ist auf einen Seed reduziert. Supersedet die Callback-Authentifizierung aus ADR-25.
ADR-28	Jedes gespeicherte Artefakt ist unter einem zufälligen Schlüssel je Löschbereich verschlüsselt, der Schlüssel existiert im Ruhezustand nur HSM-gewickelt, und Art.-17-Löschung ist die protokollierte Vernichtung dieser Kopien auf beiden Föderationsinstanzen.
ADR-29	Die organisatorische Autorität einer Handlungsvollmacht ist an ihren Aussteller gebunden; die Rolle Integration Manager umfasst Integrationen, nicht Zugriff.
ADR-30	Ein Artefakt, das die authentifizierte Entschlüsselung nicht besteht, ist ein negatives Prüfergebnis, kein Serverfehler; das Verifikationsergebnis benennt seine Fehlerklasse direkt.
ADR-31	Ausstellervertrauen ist zweckgebunden (<code>login</code> / <code>peer</code> / <code>pid</code>), an Organisationen gebunden und über einen deklarierten Mechanismus aufgelöst; die Handlungsvollmacht hinter einer Signatur reist mit und wird von der Gegenseite geprüft.
ADR-32	Kontinuierliches Compliance-Monitoring läuft als geplanter Sweep über ein eigenes Kommando statt als Schleife im Server, mit einem Risiko-Register zur Deduplizierung der Alarmer.

ADR	These und Status
ADR-33	Das Zielsystem klassifiziert die Vertragserfüllung zur Laufzeit; das DCS zeichnet die gemeldeten Urteile auf, attribuiert sie zur ODRL-Regel und auditiert sie, statt sie selbst abzuleiten. Supersedet das Ausführungs-/KPI-Moment aus ADR-11.
ADR-34	Eine Statusliste wird vom Aussteller signiert und bedient, dessen Credential sie registert, und genauso verifiziert wie das Credential selbst; eine unsignierte Statusliste wird unter keiner Konfiguration akzeptiert.
ADR (OCM-W)	VC-Signierung läuft direkt über das HSM, nicht über die OCM-W-Dienste. Einziger unnummerierter ADR der Reihe.

Backend-interne Reihe

Ältere, backend-lokale Entscheidungen unter `docs/backend/en/decisions/`. Sie beschreiben Grundmuster, auf denen die Projektreihe aufsetzt.

ADR	These
0001	CQRS-Aufteilung je Fachdomäne (command/query/db/datatype/event).
0002	Design-first-API mit generiertem Transport.
0003	Event-sourced Audit-Trail mit Hash-Ketten je Ressource, verfeinert durch ADR-16.
0004	did:web-Identität mit eIDAS-Zertifikatskette als Vertrauensanker.
0005	Ein Schreiber je Vertrag (die Origin-Instanz); die Transportform ist durch ADR-13 abgelöst.
0006	Versionierungsstrategien für Vorlagen und Verträge.
0007	Optimistische Nebenläufigkeitskontrolle über den Änderungszeitstempel.